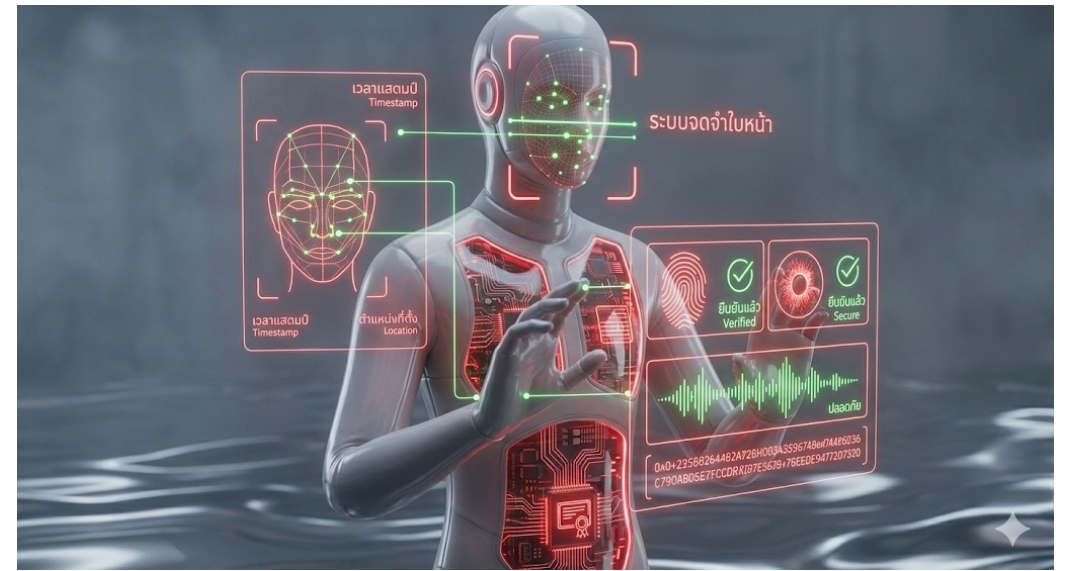


การพัฒนาาระบบสารสนเทศ อย่างปลอดภัย

วันที่ 2 : การจัดการตัวตนและการเข้าถึงข้อมูล



IDENTIFICATION AND AUTHENTICATION FAILURES



- เป็นด่านแรกของการเข้าถึงระบบ หากด่านนี้พัง
- ผู้โจมตีจะสามารถสวมรอยเป็นผู้ใช้งาน (**Impersonation**) หรือผู้ดูแลระบบได้ทันที
- มีความถี่ในการตรวจพบสูงเป็นอันดับต้น ๆ ในการทดสอบเจาะระบบ (**Penetration Testing**)



A07:2021-IDENTIFICATION AND AUTHENTICATION FAILURES

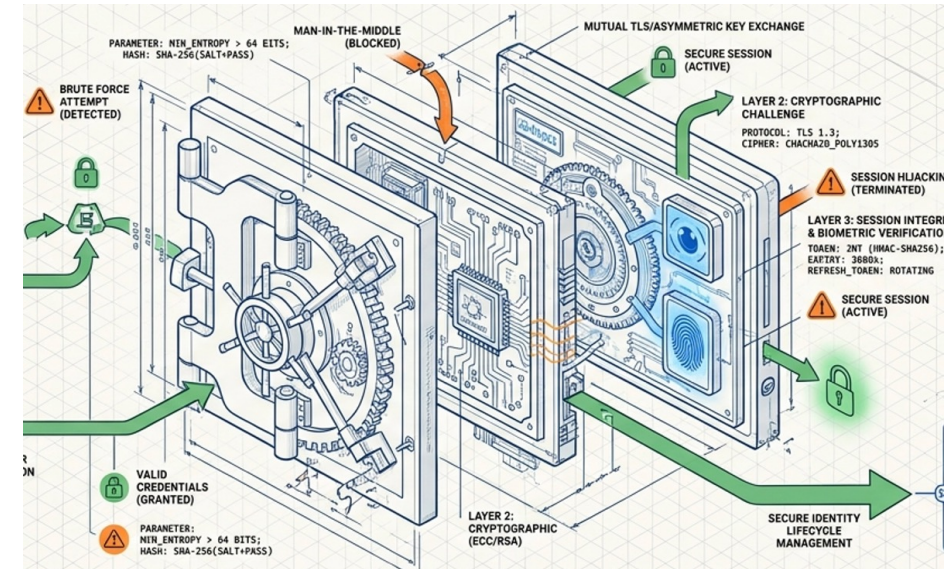


- การยืนยันตัวตนของผู้ใช้งาน (**Authentication**) การตรวจสอบความถูกต้อง และการจัดการเซสชัน (**Session Management**) เป็นเรื่องที่สำคัญมากในการป้องกันการโจมตีที่เกี่ยวข้องกับการเข้าสู่ระบบ
- หากแอปพลิเคชันมีจุดอ่อนในระบบยืนยันตัวตน อาจส่งผลให้เกิดปัญหาดังนี้
 - การอนุญาตให้ใช้รหัสผ่านที่คาดเดาง่าย: ระบบไม่มีการบังคับความซับซ้อนของรหัสผ่าน
 - ช่องโหว่จากการใช้รหัสผ่านซ้ำ: ผู้ใช้งานนำรหัสผ่านที่เคยรู้ไว้จากที่อื่นมาใช้ (**Credential Stuffing**)
 - การจัดการเซสชันที่ไม่ปลอดภัย: เช่น เซสชันไม่มีวันหมดอายุ หรือ **Token** ไม่ถูกลบหลังจาก **Log out**
 - ขาดการยืนยันตัวตนแบบหลายปัจจัย (**MFA**): ทำให้ผู้โจมตีเข้าถึงระบบได้เพียงแต่รู้รหัสผ่านอย่างเดียว

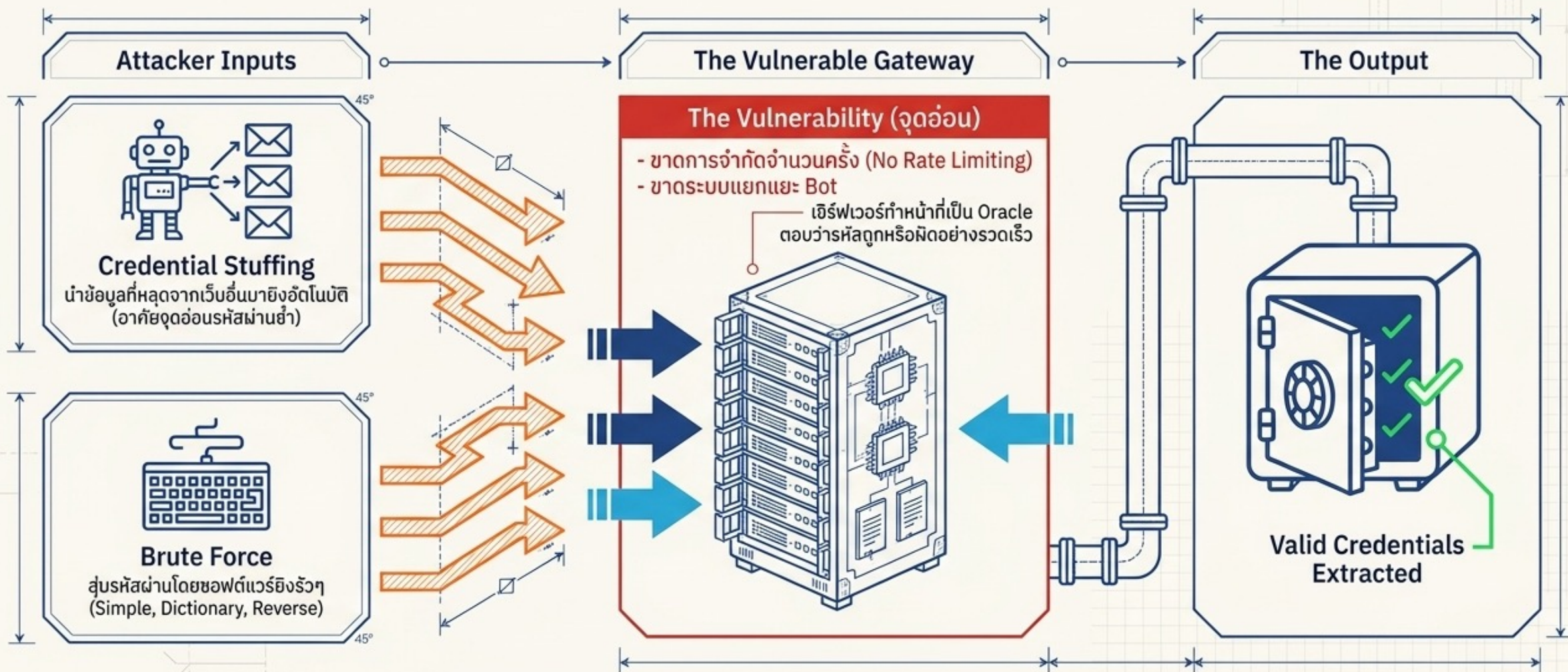


รายละเอียดช่องโหว่ด้านการระบุตัวตนและการยืนยันตัวตน (A07:2021)

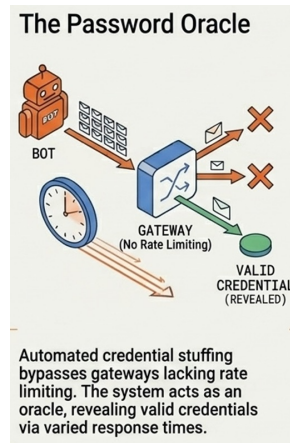
- ยอมให้มีการโจมตีแบบอัตโนมัติ เช่น **Credential Stuffing**
- ยอมให้มีการโจมตีแบบ **Brute Force**
- อนุญาตให้ใช้รหัสผ่านที่คาดเดาง่ายหรือรหัสผ่านเริ่มต้น
- ใช้กระบวนการกู้คืนรหัสผ่านที่อ่อนแอหรือไม่ปลอดภัย
- จัดเก็บรหัสผ่านแบบข้อความธรรมดา (**Plain Text**) หรือแฮชที่อ่อนแอ
- ขาดระบบการยืนยันตัวตนแบบหลายปัจจัย (**MFA**)
- เปิดเผย **Session Identifier (ID)** บน URL
- นำ **Session ID** เดิมมาใช้ซ้ำหลังจากล็อกอินสำเร็จ
- การยกเลิกเซสชัน (**Invalidation**) ไม่ถูกต้อง



การโจมตีด้านหน้า: เมื่อประตูหน้ากลายเป็นเครื่องพยากรณ์รหัสผ่าน



ยอมให้มีการโจมตีแบบอัตโนมัติ เช่น **CREDENTIAL STUFFING**

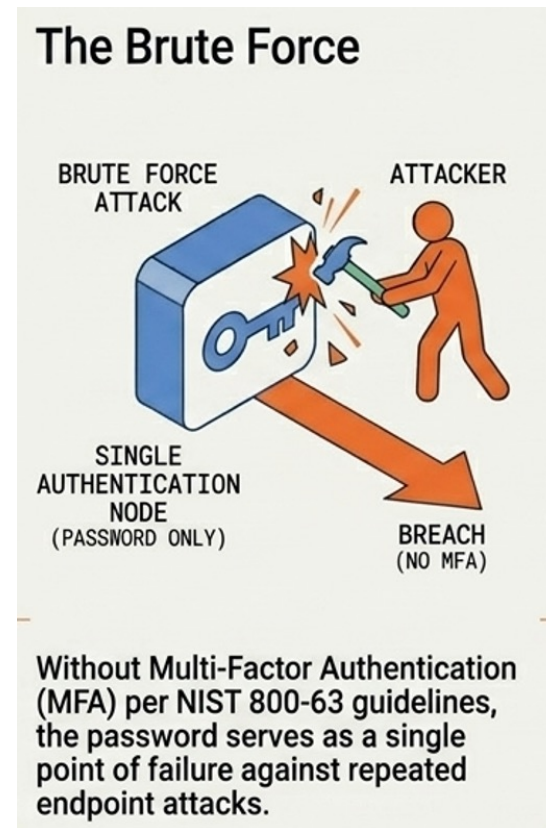


- คือการโจมตีทางไซเบอร์ที่ผู้โจมตีนำ "รายชื่อผู้ใช้งาน (**Username**) และ รหัสผ่าน (**Password**)" ที่หลุดออกมาจากเว็บไซต์หนึ่ง (**Data Breach**) ไปพยายามล็อกอินเข้าสู่เว็บไซต์หรือแอปพลิเคชันอื่น ๆ แบบอัตโนมัติ
- ทำไมการโจมตีนี้ถึงได้ผล?
 - พฤติกรรมผู้ใช้: คนส่วนใหญ่มักใช้ รหัสผ่านชุดเดียวกัน ในหลาย ๆ เว็บไซต์ (เช่น ใช้รหัสเดียวกันทั้ง **Facebook, Email** และเว็บธนาคาร)
 - ความง่าย: เมื่อมีข้อมูลหลุดจากเว็บ **A** ผู้โจมตีก็แค่ใช้บอท (**Bot**) ลองเอาข้อมูลนั้นไปยิงใส่เว็บ **B, C, D** เรื่อย ๆ จนกว่าจะเจออันที่เข้าได้
- ลักษณะของ "การยอมให้โจมตี" (**Vulnerability**)
 - **ไม่มีระบบป้องกัน Bot:** ระบบไม่สามารถแยกแยะได้ว่าใครคือคน ใครคือโปรแกรมอัตโนมัติที่กำลังลองล็อกอินรั่ว ๆ
 - **ไม่มี Rate Limiting:** ยอมให้มีการพยายามล็อกอินผิดพลาดได้ไม่จำกัดครั้งจาก **IP** เดียวกัน หรือในช่วงเวลาสั้น ๆ
 - **ขาด MFA:** หากระบบมีเพียงแค่รหัสผ่านชั้นเดียว เมื่อผู้โจมตีได้รหัสที่ถูกต้องไป ก็สามารถยึดบัญชี (**Account Takeover**) ได้ทันที



ยอมให้มีการโจมตีแบบ **BRUTE FORCE**

- วิธีการโจมตีแบบ "การลองผิดลองถูก" (**Trial and Error**) โดยผู้โจมตีจะใช้ซอฟต์แวร์อัตโนมัติในการสุ่มส่ง "รหัสผ่าน" (**Password**) หรือ "กุญแจเข้ารหัส" ไปเรื่อย ๆ จนกว่าจะเจอชุดที่ถูกต้อง
- ระบบจะถือว่ามิช้องโหว่นี้หาก
 - ไม่มีการจำกัดจำนวนครั้ง (**No Login Throttling**)
 - ไม่มีระบบตรวจจับ **Bot**
 - รหัสผ่านสั้นหรือคาดเดาง่าย
- ประเภทของ **Brute Force** ที่พบบ่อย
 - **Simple Brute Force**
 - **Dictionary Attack**
 - **Reverse Brute Force**



อนุญาตให้ใช้รหัสผ่านที่คาดเดาง่ายหรือรหัสผ่านเริ่มต้น

- หนึ่งในประตูด่านใหญ่ที่เปิดให้แฮกเกอร์เข้าสู่ระบบได้โดยแทบไม่ต้องออกแรง
- รหัสผ่านที่ตั้งมาจากโรงงานหรือมาพร้อมกับซอฟต์แวร์ตอนติดตั้งครั้งแรก เช่น
 - Admin / Admin
 - Root / 1234
 - Guest / Guest
- รหัสผ่านที่คาดเดาง่าย (**Weak Passwords**)
 - หักยอติดนิยม: 12345678, **password**, **qwerty**
 - ข้อมูลส่วนตัว: วันเกิด, เบอร์โทรศัพท์, ชื่อเล่น, ชื่อบริษัท
 - คำศัพท์ในพจนานุกรม: คำทั่วไปที่ไม่มีตัวเลขหรือสัญลักษณ์ผสม
- ทำไมระบบถึง “อนุญาต” ให้ใช้ (**Vulnerability**)?
 - ไม่มี **Password Policy**
 - ไม่มีระบบตรวจสอบ **Common Passwords**
 - ไม่บังคับเปลี่ยนรหัสผ่าน



ใช้กระบวนการกู้คืนรหัสผ่านที่อ่อนแอหรือไม่ปลอดภัย

- ช่องโหว่ที่แฮกเกอร์มักใช้ "ทางลัด" เข้าสู่ระบบโดยไม่ต้องเดารหัสผ่านเดิม แต่ไปหลอกระบบในขั้นตอนการลิ้มรสรหัสผ่านแทน
- ลักษณะของกระบวนการที่ "อ่อนแอ"
 - คำถามเพื่อความปลอดภัย (**Security Questions**)
 - คุณชอบสีอะไร? "สัตว์เลี้ยงตัวแรกชื่ออะไร?" หรือ "คุณแม่นามสกุลเดิมว่าอะไร?"
 - การส่งรหัสผ่านใหม่เป็นข้อความธรรมดา (**Plain Text**)
 - ลิงก์ **Reset** ที่ไม่มีวันหมดอายุ

หาได้ง่ายมากจาก **Social Media** หรือการทำ **Social Engineering**

ระบบส่งรหัสผ่านเดิมกลับมาทางอีเมลแทนที่จะส่งลิงก์สำหรับ **Reset**

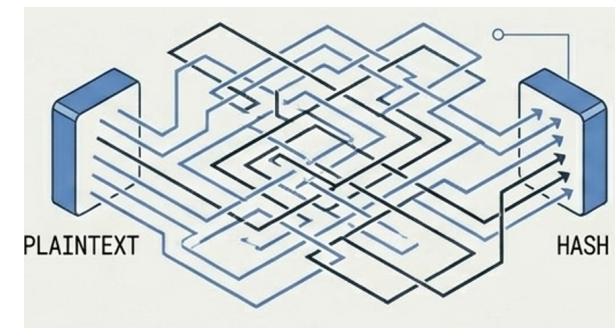


- ลิงก์ที่ส่งไปทางอีเมลสามารถใช้เมื่อไหร่ก็ได้ หรือเดารูปแบบ **URL** ได้ง่าย



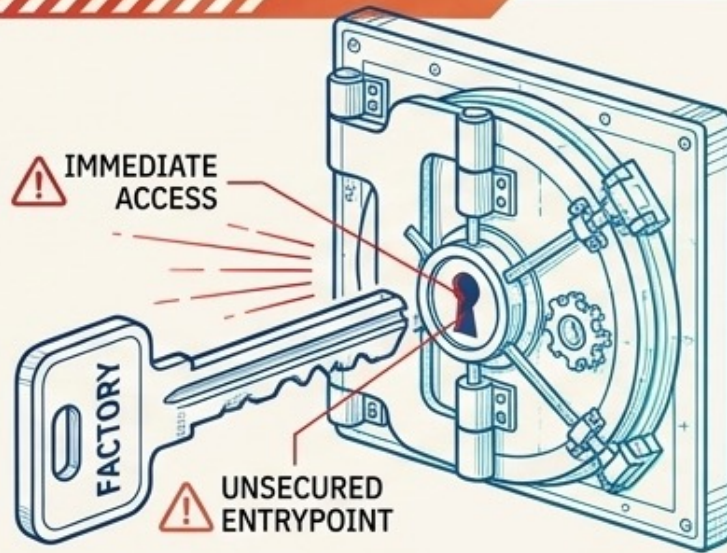
จัดเก็บรหัสผ่านแบบข้อความธรรมดา (PLAIN TEXT) หรือแฮชที่อ่อนแอ

- ผิดพลาดระดับรุนแรงในเชิงเทคนิค เพราะหากฐานข้อมูลรั่วไหล แฮกเกอร์จะได้รหัสผ่านของทุกคนไปทันทีโดยไม่ต้องถอดรหัส
- เก็บแบบข้อความธรรมดา (Plain Text)
 - ถ้าผู้ใช้ตั้งรหัสว่า **Win1234** ในตารางฐานข้อมูลก็จะเป็นคำว่า **Win1234** เลย
 - ใครก็ตามที่เข้าถึงฐานข้อมูลได้ (ไม่ว่าจะแฮกเกอร์หรือพนักงานที่ประสงค์ร้าย) จะเห็นรหัสผ่านของทุกคน และนำไปใช้ล็อกอินเว็บอื่น ๆ ได้ทันที
- การใช้แฮชที่อ่อนแอ (Weak Hashing)
 - หากใช้อัลกอริทึมที่เก่าเกินไป เช่น **MD5** หรือ **SHA-1** จะถือว่าอ่อนแอ
 - แฮกเกอร์สามารถใช้เทคนิค **Rainbow Tables** (ตารางที่คำนวณค่าแฮชไว้ล่วงหน้า) หรือการใช้ **GPU** แรง ๆ สุ่มแก้แฮช (**Brute-force hash**) เพื่อหารหัสผ่านจริงได้ภายในเวลาไม่กี่วินาที
- ทำไมถึงเป็นช่องโหว่ (Vulnerability)?
 - **Lack of Salting** ไม่มีการเพิ่มค่า **Salt** (ชุดข้อมูลสุ่ม) ลงไปก่อน **Hash** ทำให้รหัสผ่านที่เหมือนกันมีค่าแฮชเหมือนกัน แฮกเกอร์จึงเดารูปแบบได้ง่าย
 - **Fast Hashing** อัลกอริทึมทั่วไปทำงานเร็วเกินไป ทำให้แฮกเกอร์สุ่มเดารหัสได้หลายล้านชุดต่อวินาที



จุดอ่อนเชิงพฤติกรรม: รหัสผ่านเริ่มต้นและกระบวนการกู้คืนที่หละหลวม

1. รหัสผ่านเริ่มต้น (Default & Weak Credentials)



- การปล่อยรหัสผ่านจากโรงงานทิ้งไว้ (เช่น Admin/Admin, Root/1234) คือประตูลูกเหล็กที่แฮกเกอร์เดินเข้าได้ทันที
- การขาด Password Policy ทำให้ผู้ใช้ตั้งรหัสเดาง่าย (เช่น ข้อมูลส่วนตัว, คำในพจนานุกรม)

FACTORY BACKDOOR

2. ช่องโหว่การกู้คืนรหัสผ่าน (Broken Recovery)



Security Questions

คำถามหาคำตอบได้ง่ายจาก Social Media (เช่น สัตว์เลี้ยง, สีที่ชอบ)

Plain Text Emails

ส่งรหัสผ่านเดิมกลับมาทางอีเมล (แสดงว่าระบบไม่ได้เข้ารหัสข้อมูล)

Endless Links

ลิงก์รีเซตรหัสผ่านที่ไม่มีวันหมดอายุ หรือเดารูปแบบ URL ได้

ขาดระบบการยืนยันตัวตนแบบหลายปัจจัย (MFA)

- เป็นจุดอ่อนที่ร้ายแรงที่สุดอย่างหนึ่งในยุคปัจจุบัน เพราะเป็นการฝากความปลอดภัยทั้งหมดไว้กับ "รหัสผ่าน" เพียงอย่างเดียว
- **MFA (Multi-Factor Authentication)** คือการใช้หลักฐานยืนยันตัวตน มากกว่า 1 อย่าง
 - Something you know
 - Something you have
 - Something you are
- ทำไมการ "ขาด MFA" ถึงอันตราย?
 - **Credential Vulnerability** หากแฮกเกอร์ได้รหัสผ่านไป ไม่ว่าจะจากการสุ่ม (**Brute Force**), การหลอกลวง (**Phishing**), หรือข้อมูลหลุดจากเว็บอื่น (**Credential Stuffing**) แฮกเกอร์จะยึดบัญชีได้ทันที **100%**
 - **No Safety Net** ไม่มีด่านที่สองคอยยับยั้งการเข้าถึงที่ผิดปกติ
- ลักษณะของช่องโหว่ (**Vulnerability**)
 - ระบบไม่มีตัวเลือก **MFA** ให้ใช้
 - **MFA** ที่ไม่ครอบคลุม มี **MFA** เฉพาะบางหน้า แต่หน้าสำคัญกลับไม่เรียกใช้
 - **MFA** ที่อ่อนแอ การส่งรหัสทางอีเมลเพียงอย่างเดียว ซึ่งหากแฮกเกอร์เข้าอีเมลได้ **MFA** ก็ไร้ความหมาย



เปิดเผย SESSION IDENTIFIER (ID) บน URL

- เป็นความเสี่ยงที่เกี่ยวข้องกับการจัดการเซสชัน ซึ่งทำให้ผู้โจมตีสามารถ “ขโมยตัวตน” ของผู้ใช้งานไปได้โดยง่าย
- เมื่อเราล็อกอินเข้าสู่ระบบ แอปพลิเคชันจะสร้าง **Session ID** (เปรียบเสมือนบัตรผ่านชั่วคราว) เพื่อให้ระบบจำได้ว่าเราเป็นใครในขณะที่ใช้งาน โดยปกติค่านี้ควรถูกเก็บไว้ใน **HTTP Cookie** ที่ปลอดภัยและมองไม่เห็นบนหน้าจอ
- แอปพลิเคชันที่มีช่องโหว่นี้จะส่งค่า **Session ID** ไปกับที่อยู่เว็บ (URL) เช่น <https://example.com/dashboard?JSESSIONID=ABC123XYZ...>
- ความเสี่ยง
 - **Browser History** **Session ID** จะถูกบันทึกไว้ในประวัติการเข้าชมเว็บ (History) ของเบราว์เซอร์ ใครที่มาใช้เครื่องต่อสามารถเห็นและนำไปใช้ได้
 - **Referer Header** หากผู้ใช้คลิกจากหน้าดังกล่าวไปยังเว็บอื่น ค่า **Session ID** จะถูกส่งติดไปยังเว็บปลายทางด้วย ทำให้เจ้าของเว็บปลายทางเห็นค่าเซสชันของเรา
 - **Shoulder Surfing** คนที่ยืนอยู่ข้างหลังสามารถมองเห็นหรือถ่ายภาพ URL ที่มี **Session ID** ได้
 - **Web Server Logs** ค่านี้ถูกบันทึกไว้ใน **Log** ของเซิร์ฟเวอร์หรือ **Proxy** ซึ่งมักจะไม่มีมาตรการเข้ารหัส



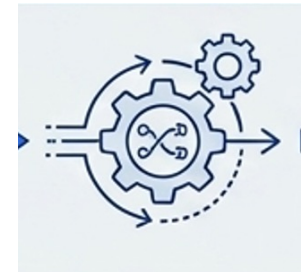
นำ SESSION ID เดิมมาใช้ซ้ำหลังจากล็อกอินสำเร็จ

- ช่องโหว่ **Session Fixation**

- ปกติ เมื่อเราเปิดหน้าเว็บ แอปพลิเคชันจะให้ **Session ID** ชุดหนึ่งมา (เป็นสถานะ "ยังไม่ได้ล็อกอิน") แต่พอเราล็อกอินสำเร็จ ระบบที่ดีควรจะยกเลิก **ID** เดิมทิ้ง และออก **ID** ชุดใหม่ให้ทันที เพื่อความปลอดภัย

- วิธีการโจมตี (Attack Scenario)

- ผู้โจมตี (**Attacker**) เข้าไปที่หน้าเว็บเพื่อเอา **Session ID** ที่ยังไม่ได้ล็อกอินมาตัวหนึ่ง (เช่น **ID: 123**)
- ส่งลิงก์ให้เหยื่อ: ผู้โจมตีส่งลิงก์ที่มี **ID** นั้นแฝงอยู่ไปให้เหยื่อ เช่น <https://example.com/login?SID=123>
- เหยื่อหลงกล: เหยื่อคลิกลิงก์และทำการล็อกอินตามปกติ
- ความผิดพลาดของระบบ: ระบบยอมให้เหยื่อใช้ **ID: 123** ต่อไปหลังจากล็อกอินสำเร็จ ทำให้ตอนนี้ **ID: 123** กลายเป็นบัตรผ่านที่เข้าถึงบัญชีของเหยื่อได้แล้ว
- ผู้โจมตีสวมรอย: เนื่องจากผู้โจมตีถือ **ID: 123** อยู่ในมือแต่แรก จึงสามารถใช้ **ID** นี้เข้าสู่ระบบในชื่อของเหยื่อได้ทันที



การยกเลิกเซสชัน (INVALIDATION) ไม่ถูกต้อง

- เปรียบเสมือนการที่เราออกจากบ้านแล้วแต่ "ลืมน็อกกุญแจ" หรือกุญแจนั้นยังสามารถใช้งานได้อยู่แม้จะเดินบัตรผ่านไปแล้ว
- **Session Invalidation**
 - กระบวนการ "ทำลาย" บัตรผ่าน (Session ID/Token) ให้สิ้นสภาพเมื่อสิ้นสุดการใช้งาน ไม่ว่าจะเป็นการกด **Log out** ด้วยตัวเอง หรือระบบตัดการทำงานเมื่อไม่มีการเคลื่อนไหว (Timeout)
- ลักษณะความผิดพลาดที่พบบ่อย (**Vulnerability**)
 - **Logout** ไม่จริง ไม่ได้ไปสั่ง **ยกเลิก ID** นั้นในฐานะข้อมูลฝั่งเซิร์ฟเวอร์ ผลคือหากแฮกเกอร์ขโมยค่า **ID** นั้นไปได้ก่อนหน้า ก็ยังสามารถใช้ **ID** นั้นเข้าสู่ระบบได้ต่อไป
 - ไม่มีระบบ **Idle Timeout** เซสชันไม่มีวันหมดอายุ หรือนานเกินไป (เช่น 30 วัน) แม้ผู้ใช้จะไม่ได้ขยับหน้าจอเลย ทำให้หากใครมาใช้เครื่องต่อก็สามารถเข้าถึงข้อมูลได้ทันที
 - **SSO Token** ไม่ถูกทำลาย ในระบบ **Single Sign-On** เมื่อ **Log out** จากแอปหนึ่งแล้ว แต่ **Token** หลักที่ใช้ล็อกอินแอปอื่น ๆ ในเครื่องยังคงใช้งานได้อยู่



กายวิภาคของการขโมยบัตรผ่าน (Session Management Failures)

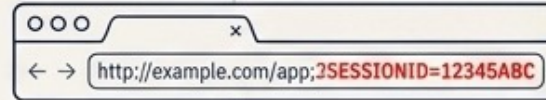
1. Session Fixation



Pre-login

ผู้โจมตีกำหนด Session ID
หลอกให้เหยื่อล็อกอิน
หากระบบใช้ ID เดิมซ้ำ
แฮกเกอร์จะสวมรอยได้ทันที

2. URL Exposure



SESSION ID LIFECYCLE

Active Session

การปล่อย Session ID โชว์บน URL
ทำให้เสี่ยงต่อการถูกบันทึกใน
Browser History หรือถูกแอบมอง
(Shoulder Surfing)

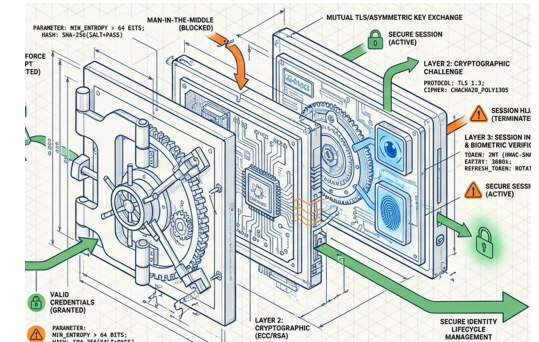
3. Invalidation Failure



Logout

เปรียบเสมือนออกบ้านแต่ลืมล็อกกุญแจ
เกิดจากระบบซ่อนหน้าเว็บแต่
ไม่ได้ยกเลิก ID ในฐานข้อมูล
หรือขาด Idle Timeout

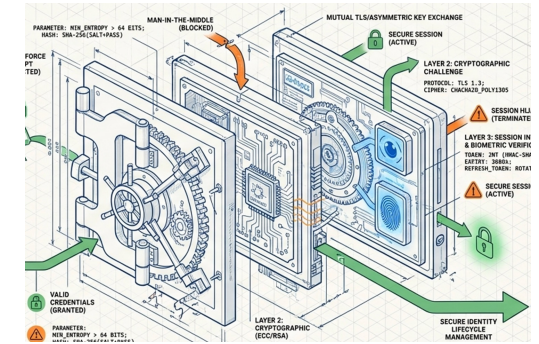
การป้องกัน



- ใช้การยืนยันตัวตนแบบหลายปัจจัย (**MFA**) ทุกที่ที่เป็นไปได้ เพื่อป้องกันการโจมตีแบบอัตโนมัติ (**Credential Stuffing**), การสุมรหัสผ่าน (**Brute Force**) และการนำรหัสผ่านที่ถูกขโมยมาใช้ซ้ำ
- ห้ามปล่อยให้มืรหัสผ่านเริ่มต้น (**Default Credentials**) ห้ามติดตั้งหรือใช้งานระบบที่ยังมืรหัสผ่านเริ่มต้นค้างอยู่ โดยเฉพาะอย่างยิ่งสำหรับบัญชีระดับผู้ดูแลระบบ (**Admin**)
- ตรวจสอบรหัสผ่านที่อ่อนแอ มีระบบตรวจสอบเมื่อผู้ใช้ตั้งค่าหรือเปลี่ยนรหัสผ่านใหม่ โดยเทียบกับรายการรหัสผ่านยอดแย่ (เช่น **Top 10,000 worst passwords**)
- ปรับนโยบายรหัสผ่านให้เป็นมาตรฐานสากล กำหนดความยาว ความซับซ้อน และนโยบายการเปลี่ยนรหัสผ่านตามแนวทางของ **NIST 800-63b** หรือนโยบายสมัยใหม่ที่เน้นความปลอดภัยที่พิสูจน์ได้จริง



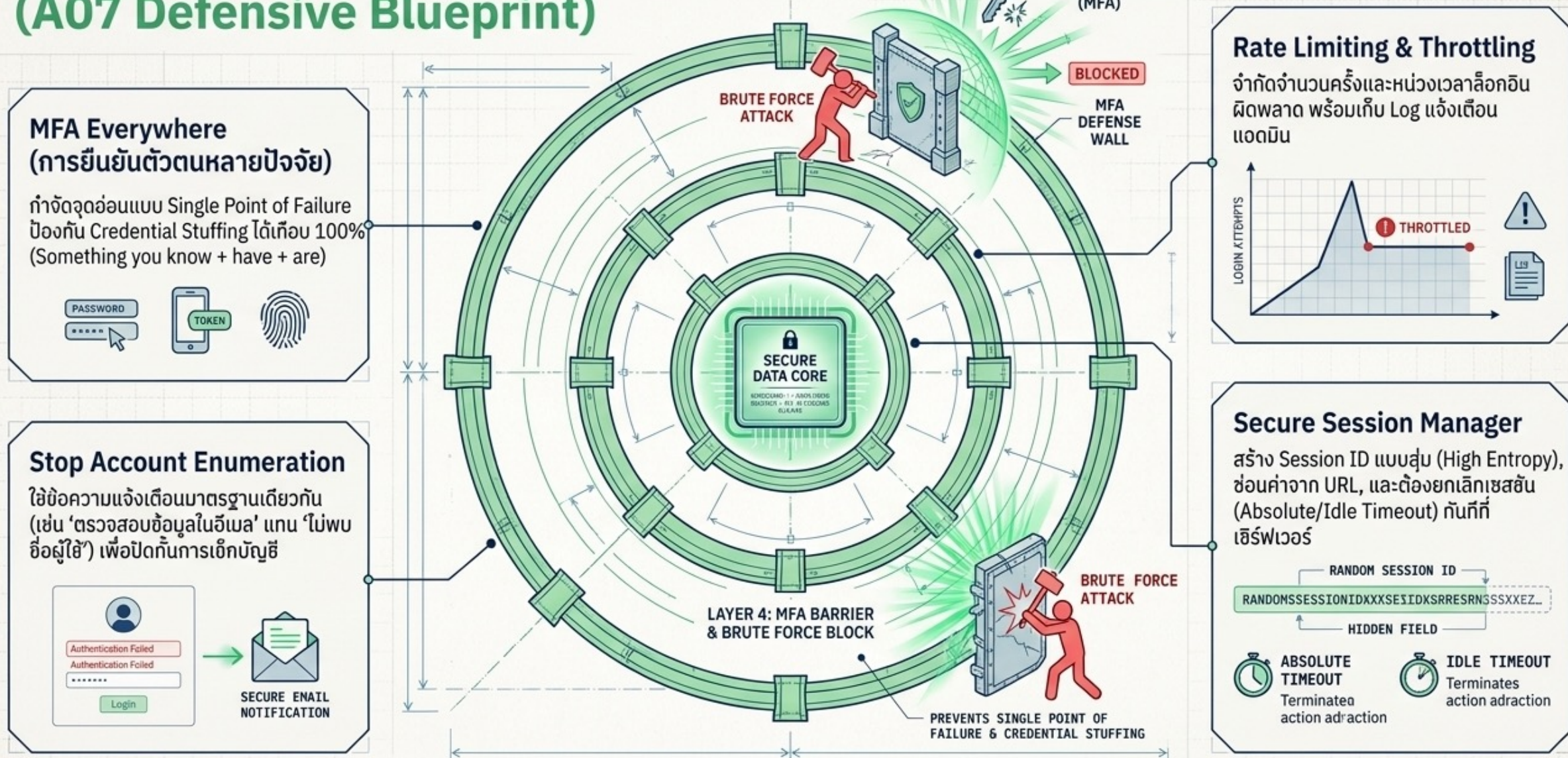
การป้องกัน (ต่อ)



- ป้องกันการสุมตรวจสอบรายชื่อบัญชี (**Account Enumeration**) ทำให้ขั้นตอนการลงทะเบียน การกู้คืนรหัสผ่าน และช่องทาง **API** มีความปลอดภัย โดยการ **ใช้ข้อความตอบกลับที่เหมือนกันในทุกกรณี** (เช่น ไม่ระบุว่า "ไม่พบชื่อผู้ใช้" แต่ให้ใช้ว่า "ตรวจสอบข้อมูลในอีเมลของท่าน")
- จำกัดหรือหน่วงเวลาเมื่อล็อกอินผิด จำกัดจำนวนครั้งหรือเพิ่มระยะเวลาหน่วงเมื่อมีความพยายามล็อกอินที่ล้มเหลว (**Rate Limiting**) แต่ต้องระวังไม่ให้กระทบจนกลายเป็นช่องทางโจมตีให้ระบบใช้งานไม่ได้ (**DoS**) พร้อมทั้งเก็บ **Log** และแจ้งเตือนผู้ดูแลระบบเมื่อตรวจพบการโจมตี
- ใช้ระบบจัดการเซสชันที่ปลอดภัยฝั่งเซิร์ฟเวอร์ ใช้ **Session Manager** มาตรฐานที่สร้าง **Session ID** ใหม่แบบสุ่มที่มีความปลอดภัยสูง (**High Entropy**) หลังจากล็อกอินสำเร็จ โดยต้องไม่โชว์ ID บน **URL**, จัดเก็บอย่างปลอดภัย และมีการยกเลิกเซสชัน (**Invalidate**) ทันทีเมื่อ **Logout**, เมื่อไม่มีการใช้งาน (**Idle**), หรือเมื่อครบกำหนดเวลาสูงสุด (**Absolute Timeout**)



การเสริมความแข็งแกร่งให้ประตูด้านหน้า (A07 Defensive Blueprint)



A02:2021 — CRYPTOGRAPHIC FAILURES



- การกำหนดความต้องการในการปกป้องข้อมูล (Data Protection Needs)
- สิ่งแรกที่ต้องทำคือการระบุความต้องการในการปกป้องข้อมูล ทั้ง **ขณะที่มีการรับส่งข้อมูล (In Transit)** และ **ขณะที่จัดเก็บอยู่ในระบบ (At Rest)** ตัวอย่างเช่น รหัสผ่าน, หมายเลขบัตรเครดิต, ประวัติการรักษาพยาบาล, ข้อมูลส่วนบุคคล และความลับทางธุรกิจ ข้อมูลเหล่านี้จำเป็นต้องได้รับการปกป้องเป็นพิเศษ โดยเฉพาะอย่างยิ่งหากข้อมูลนั้นอยู่ภายใต้กฎหมายคุ้มครองข้อมูลส่วนบุคคล เช่น **GDPR** ของสหภาพยุโรป หรือข้อบังคับเฉพาะทาง เช่น มาตรฐานความปลอดภัยข้อมูลบัตรเครดิต **PCI DSS** สำหรับข้อมูลทั้งหมดที่กล่าวมานี้
- มีแนวทางปฏิบัติดังนี้



มีข้อมูลใดบ้างที่ถูกรับส่งโดยไม่มีการเข้ารหัส (**CLEAR TEXT**) หรือไม่?

- ประเด็นนี้ครอบคลุมถึงโปรโตคอลต่าง ๆ เช่น **HTTP, SMTP, FTP**
- รวมถึงการใช้การอัปเดตความปลอดภัยด้วย **STARTTLS**
- การรับส่งข้อมูลผ่านอินเทอร์เน็ตภายนอกนั้นมีความเสี่ยงสูง
- ขณะเดียวกัน **ต้องตรวจสอบการรับส่งข้อมูลภายในทั้งหมดด้วย**
 - เช่น ข้อมูลที่วิ่งอยู่ระหว่าง **Load Balancer, Web Server** หรือระบบหลังบ้าน (**Back-end Systems**)
- **HTTP / FTP / SMTP:** หากใช้งานแบบดั้งเดิม ข้อมูลจะไม่มีการเข้ารหัสเลย
- **STARTTLS:** เป็นการอัปเดตการเชื่อมต่อจากไม่มีรหัสให้เป็นแบบเข้ารหัส แต่ถ้การอัปเดตนี้ล้มเหลว (และระบบไม่ตัดการทำงาน) ข้อมูลจะถูกส่งออกไปแบบ **Clear Text** กันที่โดยที่ผู้ใช้ไม่รู้ตัว



มีการใช้อัลกอริทึมหรือโปรโตคอลการเข้ารหัสที่ เก่า หรืออ่อนแอ ทั้งที่ถูกตั้งค่ามาเป็นค่าเริ่มต้น (DEFAULT) หรือที่ยังค้างอยู่ในโค้ดรุ่นเก่า (OLDER CODE) หรือไม่?

- ในโลกของ **Cybersecurity** "ความปลอดภัยมีวันหมดอายุ" อัลกอริทึมที่เคยว่ากันว่าปลอดภัยในเมื่อ 10 ปีก่อน ในปัจจุบันอาจถูกเจาะได้ง่ายๆ ด้วยคอมพิวเตอร์ทั่วไป
- อัลกอริทึมที่ "อ่อนแอ" และไม่ควรใช้แล้ว
 - **MD5 / SHA-1:** สำหรับการทำ **Password Hashing** สองตัวนี้ถือว่าอันตรายมาก ฮกเกอร์สามารถใช้เทคนิคที่เรียกว่า **Collision Attack** หรือใช้ตารางคำนวณล่วงหน้า (**Rainbow Tables**) หาค่ารหัสจริงได้ในพริบตา
 - **DES / 3DES:** สำหรับการเข้ารหัสข้อมูล (**Encryption**) ปัจจุบันถูกแทนที่ด้วย **AES** ที่แข็งแกร่งกว่ามาก
 - **RC4:** โปรโตคอลการเข้ารหัสที่มักพบในระบบเก่าๆ ซึ่งมีจุดโหว่ที่ทำให้แฮกเกอร์ถอดรหัสข้อมูลได้
- โปรโตคอลการรับส่งข้อมูลที่ล้าสมัย
 - **SSL v2 / v3:** ล้าสมัยและมีช่องโหว่ร้ายแรง
 - **TLS 1.0 / 1.1:** ปัจจุบันมาตรฐานความปลอดภัยส่วนใหญ่ (รวมถึงเบราว์เซอร์สมัยใหม่) บังคับให้ใช้ **TLS 1.2** หรือ 1.3 เท่านั้น
- ปัญหาของ "**Legacy Code**" (โค้ดเก่า)
 - เรียกใช้ฟังก์ชันการเข้ารหัสแบบเก่า ทำให้กลายเป็น "**จุดอ่อนที่ถูกลืม**" (**Hidden Weak Point**) ของทั้งระบบ



มีการใช้ **กุญแจเข้ารหัสแบบดำเริ่มต้น** , มีการสร้างหรือใช้กุญแจที่ **อ่อนแอ** ช้าหรือไม่? ระบบขาดการจัดการกุญแจหรือการ **ผลิตเปลี่ยนกุญแจ** ที่ถูกต้องหรือไม่? และมีการเฝ้าเก็บกุญแจเข้ารหัสไว้ใน **SOURCE CODE REPOSITORIES** (เช่น **GITHUB, GITLAB**) หรือไม่?

- เปรียบเสมือนการที่เราติดตั้งแม่กุญแจเราดาแพงไว้ที่บ้าน แต่เรากลับเก็บลูกกุญแจไว้ใต้พรมเช็ดเท้า หรือใช้กุญแจดอกเดิมซ้ำ ๆ มานานหลายปี
- **Default & Weak Keys** (กุญแจเริ่มต้นและกุญแจที่อ่อนแอ)
 - **Default Keys:** ซอฟต์แวร์หลายตัวมาพร้อมกุญแจทดสอบ (**Test Keys**) หากเราไม่เปลี่ยนตอนนำไปใช้งานจริง แฮกเกอร์ที่รู้กุญแจนี้จะสามารถถอดรหัสข้อมูลได้ทันที
 - **Weak Keys:** กุญแจที่สั้นเกินไปหรือถูกสร้างจากระบบสุ่มที่ไม่มีคุณภาพ ทำให้แฮกเกอร์เดารูปแบบกุญแจได้
- **Proper Key Management & Rotation** (การจัดการและการผลิตเปลี่ยน)
 - **Missing Rotation:** หากเราใช้กุญแจเดิมนานเกินไป (เช่น 5-10 ปี) หากกุญแจนั้นเคยหลุดไปในอดีต แฮกเกอร์ก็จะยังคงใช้มันเข้าถึงข้อมูลได้ตลอดไป การผลิตเปลี่ยนกุญแจ (**Rotation**) เป็นระยะจะช่วยลดความเสียหายหากกุญแจหลุดเก๋ารั่วไหล
 - **Key Management:** การเก็บกุญแจไว้ในที่ที่ปลอดภัย (เช่น **Key Vault** หรือ **Hardware Security Module - HSM**) แทนการเก็บเป็นไฟล์ธรรมดาในเซิร์ฟเวอร์
- **Hardcoded Keys in Source Code** (การฝังกุญแจไว้ในโค้ด)



ระบบ ไม่ได้บังคับใช้การเข้ารหัส หรือไม่? ตัวอย่างเช่น มีการขาดการตั้งค่า **HTTP SECURITY HEADERS** (คำสั่งด้านความปลอดภัยสำหรับเบราว์เซอร์) ที่สำคัญไปหรือไม่?

- แม้ว่าเว็บไซต์จะใช้ **HTTPS** (มีรูปกุญแจ) แต่ถ้าขาดการตั้งค่า "คำสั่งกำกับความปลอดภัย" (**Security Headers**) ที่ถูกต้อง เบราว์เซอร์ก็อาจจะยังยอมให้เกิดช่องโหว่บางอย่างได้
- **Security Headers** ที่สำคัญและมักถูกลืม
 - **HSTS (HTTP Strict Transport Security):**
 - หน้าที่: สั่งให้เบราว์เซอร์เข้าเว็บนี้ผ่าน **HTTPS** เท่านั้น ในอนาคต
 - ความเสี่ยง: หากไม่มีตัวนี้ แฮกเกอร์อาจใช้เทคนิค **SSL Stripping** เพื่อบีบให้เบราว์เซอร์ย้อนกลับไปใช้ **HTTP** ธรรมดาเพื่อดักดูข้อมูลได้
 - **Content Security Policy (CSP):**
 - หน้าที่: กำหนดว่าสคริปต์ (**JavaScript**) หรือรูปภาพชุดไหนที่ "อนุญาต" ให้รันบนเว็บเราได้บ้าง
 - ความเสี่ยง: ช่วยป้องกันการโจมตีแบบ **XSS (Cross-Site Scripting)** ไม่ให้โค้ดแปลกปลอมมาขโมยข้อมูลในหน้าเว็บเรา
 - **X-Frame-Options:**
 - หน้าที่: ป้องกันไม่ให้เว็บเราถูกนำไปเปิดใน **<iframe>** บนเว็บอื่น
 - ความเสี่ยง: ป้องกันการโจมตีแบบ **Clickjacking** (การสร้างปุ่มปลอมมาซ้อนทับปุ่มจริงเพื่อหลอกให้ผู้ใช้คลิก)
 - **X-Content-Type-Options: nosniff:**
 - หน้าที่: สั่งไม่ให้เบราว์เซอร์ "เดา" ประเภทของไฟล์เอง
 - ความเสี่ยง: ป้องกันแฮกเกอร์หลอกระบบให้รันไฟล์สคริปต์ที่ปลอมแปลงมาในรูปแบบไฟล์รูปภาพ



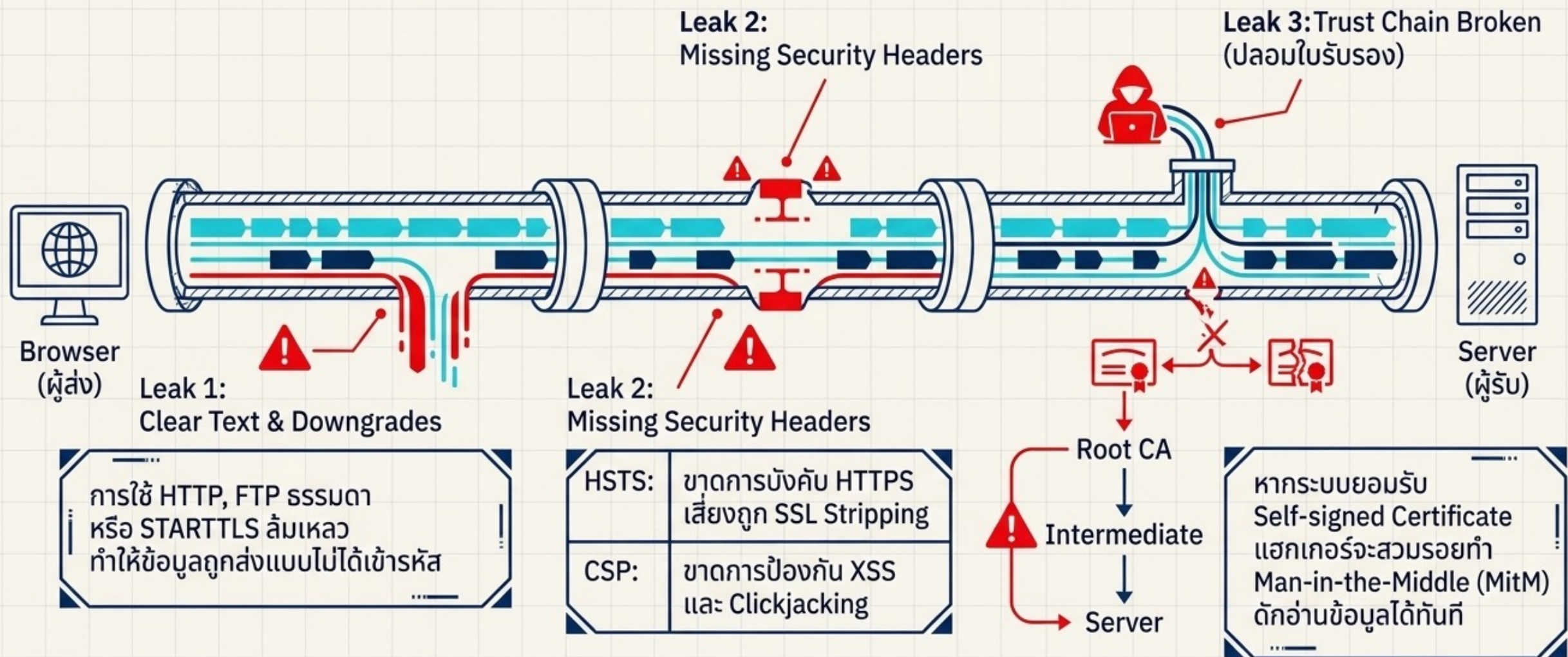
ใบรับรองของเซิร์ฟเวอร์ (**SERVER CERTIFICATE**) ที่ได้รับมา และ สายลำดับความน่าเชื่อถือ (**TRUST CHAIN**) ได้รับการตรวจสอบความถูกต้องอย่างเหมาะสมหรือไม่?

- เวลาที่เราเข้าเว็บ **HTTPS** เซิร์ฟเวอร์จะส่ง “ใบรับรองดิจิทัล” (**Digital Certificate**) มาให้เครื่องเราเปรียบเสมือนบัตรประชาชนของเว็บไซต์นั้น ๆ แต่แค่มีใบรับรองยังไม่พอ ระบบต้องตรวจสอบให้แน่ใจว่าบัตรนั้น เป็นของจริงและเชื่อถือได้
- การตรวจสอบความถูกต้อง (**Proper Validation**) บราวเซอร์จะเช็คเงื่อนไขพื้นฐาน 3 อย่าง
 - ยังไม่หมดอายุ: ใบรับรองต้องอยู่ในช่วงวันที่ที่กำหนด
 - ชื่อตรงกัน: ชื่อโดเมนในใบรับรอง (เช่น **xyz.co.th**) ต้องตรงกับเว็บที่เรากำลังเข้าชมจริง
 - ไม่ถูกเพิกถอน: ใบรับรองนั้นต้องไม่ถูกยกเลิก (**Revoked**) ก่อนกำหนดโดยผู้ออกใบรับรอง
- สายลำดับความน่าเชื่อถือ (**Trust Chain**) ต้องมี “ผู้ออกใบรับรอง” (**Certificate Authority - CA**) เป็นคนเซ็นชื่อรับรองให้ กิดขึ้นเป็นทอด ๆ ดังนี้
 - **Root CA**: องค์กรสูงสุดที่บราวเซอร์ทั่วโลกเชื่อถือ (เปรียบเหมือนรัฐบาล)
 - **Intermediate CA**: องค์กรตัวกลางที่ได้รับอำนาจจาก **Root CA**
 - **Server Certificate**: ใบรับรองของเว็บไซต์เรา (เปรียบเหมือนบัตรประชาชนที่อำเภอออกให้)
- ความเสี่ยงหาก “ไม่ได้ตรวจสอบอย่างเหมาะสม” → **Man-in-the-Middle (MitM)**
 - แฮกเกอร์สร้างใบรับรองปลอมขึ้นมาเอง (**Self-signed**)
 - แฮกเกอร์แทรกกลางการเชื่อมต่อและส่งใบรับรองปลอมนั้นให้เหยื่อ
 - หากแอปพลิเคชันไม่เช็ก **Trust Chain** มันจะยอมรับใบรับรองปลอมนั้น และแฮกเกอร์จะเห็นข้อมูลที่ส่งไปมาทั้งหมดได้ทันที



ท่ส่งข้อมูลที่รั่วไหล

(In Transit & Certificate Failures)



มีการละเลย, นำกลับมาใช้ซ้ำ หรือสร้าง **INITIALIZATION VECTORS (IV)** ที่ไม่ปลอดภัยเพียงพอสำหรับโหมดการเข้ารหัสที่ใช้หรือไม่? มีการใช้โหมดการทำงานที่ไม่ปลอดภัย เช่น **ECB** หรือไม่? และมีการใช้การเข้ารหัสแบบธรรมดาในขณะที่การเข้ารหัสแบบยืนยันความถูกต้องได้ มีความเหมาะสมมากกว่าหรือไม่?

- **Initialization Vector (IV)** คืออะไร?
- **IV** คือ “ค่าเริ่มต้นแบบสุ่ม” ที่ใช้ผสมกับข้อมูลก่อนเข้ารหัส เพื่อให้แน่ใจว่า “ถ้าเราเข้ารหัสค่าเดิมสองครั้ง ผลลัพธ์ที่ได้จะไม่เหมือนเดิม”
 - ความเสี่ยง: หากเรา ใช้ **IV** ซ้ำ (**Reuse**) หรือใช้ค่าที่คาดเดาได้ (เช่น 000000) แฮกเกอร์จะสามารถวิเคราะห์รูปแบบของข้อมูลและถอดรหัสออกมาได้โดยไม่ต้องรู้กุญแจ (**Key**) เลย
- โหมด **ECB (Electronic Codebook)** - “โหมดอันตราย”
 - **ECB** เป็นโหมดการเข้ารหัสที่ง่ายที่สุด แต่ ไม่ปลอดภัยที่สุด เพราะมันจะเข้ารหัสข้อมูลที่ละบล็อกแยกกันโดยไม่มีการสุ่ม
 - ผลลัพธ์: หากข้อมูลต้นฉบับมีส่วนที่ซ้ำกัน (เช่น พื้นหลังของรูปภาพ หรือข้อความที่ซ้ำกัน) ผลลัพธ์ที่เข้ารหัสแล้วก็จะยังคงเห็น “รูปแบบ” เดิมอยู่ แฮกเกอร์จึงสามารถเดาข้อมูลได้จากรูปร่างของมัน
- **Authenticated Encryption** (การเข้ารหัสที่ยืนยันความถูกต้องได้)
 - การเข้ารหัสแบบปกติช่วยแค่ “ไม่ให้ใครอ่านออก” (**Confidentiality**) แต่ไม่ได้ป้องกัน “การถูกแก้ไขระหว่างทาง” (**Integrity**)
 - **Authenticated Encryption** (เช่น โหมด **AES-GCM**): จะให้ทั้งความลับและมี “ตัวตรวจสอบ” (**Tag**) แนบไปด้วย หากแฮกเกอร์แอบแก้ไขข้อมูลแม้แต่บิตเดียว ระบบจะรู้ทันทีและปฏิเสธข้อมูลนั้น
 - เปรียบเทียบ: เหมือนการส่งจดหมายที่นอกจากจะใส่ซองปิดผนึกแล้ว (**Encryption**) ยังมีตราประทับซีฟิงที่เซ็นกำกับไว้ด้วย (**Authentication**) ถ้าตราประทับแตก แปลว่ามีคนแอบแก้ไขเนื้อหาข้างใน



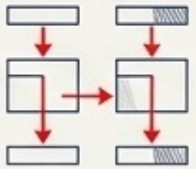
รอยร้าวในห้องนิรภัย (At Rest & Legacy Failures)

Deprecated Algorithms (เทคโนโลยีหมดอายุ)

อัลกอริทึมอย่าง DES, 3DES, RC4
อ่อนแอเกินไปและถูกเจาะได้ด้วย
คอมพิวเตอร์ทั่วไป

Dangerous Modes (ECB)

โหมด Electronic Codebook (ECB)
เข้ารหัสทีละบล็อกโดยไม่สลับ
ผลลัพธ์ยังคงเห็น รูปแบบ
เดิมของข้อมูลต้นฉบับ
ทำให้เดาได้



Sensitive Data Caching

การไม่ปิดกั้น Cache ทำให้ข้อมูลอ่อน
ไหวตกค้างในคอมพิวเตอร์สาธารณะ

วิธีแก้: บังคับใช้ Header
Cache-Control: no-store เสมอ

มีการนำ รหัสผ่าน มาใช้เป็น กุญแจเข้ารหัส โดยตรง โดยที่ไม่มีการผ่านกระบวนการ ฟังก์ชันการสร้างกุญแจจากรหัสผ่าน (**PASSWORD-BASED KEY DERIVATION FUNCTION - PBKDF**) หรือไม่?

- **Developer** นำ "รหัสผ่านที่ผู้ใช้พิมพ์มา" ไปใส่ในช่องกุญแจ (**Key**) ของอัลกอริทึมเข้ารหัส (เช่น **AES**) ทันที ซึ่งเป็นเรื่องที่อันตรายมาก
- ทำไมถึงห้ามใช้รหัสผ่านเป็นกุญแจตรง ๆ?
 - ความยาวไม่พอ: อัลกอริทึมอย่าง **AES-256** ต้องการกุญแจที่ยาว 256 บิตพอดีและมีความสุ่มสูง แต่รหัสผ่านของมนุษย์มักสั้นและคาดเดาได้ (**Low Entropy**)
 - ความเร็วในการสุ่ม: หากใช้รหัสผ่านตรง ๆ แฮกเกอร์จะสามารถใช้คอมพิวเตอร์ความเร็วสูงสุ่มเดา (**Brute-force**) รหัสผ่านนั้นได้เร็วมาก เพราะคอมพิวเตอร์ดำเนินการเข้ารหัสได้หลายล้านครั้งต่อวินาที
- **PBKDF** ตัวช่วยทำความสะอาดและยืดรหัส
 - **Salting**: เติมข้อมูลสุ่มลงไปเพื่อให้รหัสผ่านที่เหมือนกันกลายเป็นกุญแจที่ต่างกัน
 - **Key Stretching (Hashing รัว ๆ)**: เป็นการสั่งให้คอมพิวเตอร์คำนวณ **Hash** ซ้ำไปซ้ำมาหลายหมื่นหรือหลายแสนครั้ง เพื่อให้การคำนวณกุญแจหนึ่งดอกใช้เวลาประมาณ 0.1 - 0.5 วินาที
 - ผลลัพธ์: แม้รหัสผ่านจะสั้น แต่กุญแจที่ได้จะยาวและสุ่มตามมาตรฐาน และที่สำคัญคือจะทำให้แฮกเกอร์สุ่มรหัสผ่านได้ช้าลงมหาศาล (เพราะการเดา 1 ครั้งต้องใช้เวลาดำเนินการนานขึ้น)
- ความเสี่ยงหากขาด **PBKDF**
 - หากแฮกเกอร์ได้ฐานข้อมูลที่เข้ารหัสไว้โดยใช้รหัสผ่านตรง ๆ เขาจะสามารถเขียนโปรแกรมสุ่มรหัสผ่านยอดนิยม (**Dictionary Attack**) เพื่อปลดล็อกข้อมูลทั้งหมดได้ภายในเวลาอันรวดเร็ว เพราะระบบไม่มี "ตัวหน่วงเวลา" ในการสร้างกุญแจ

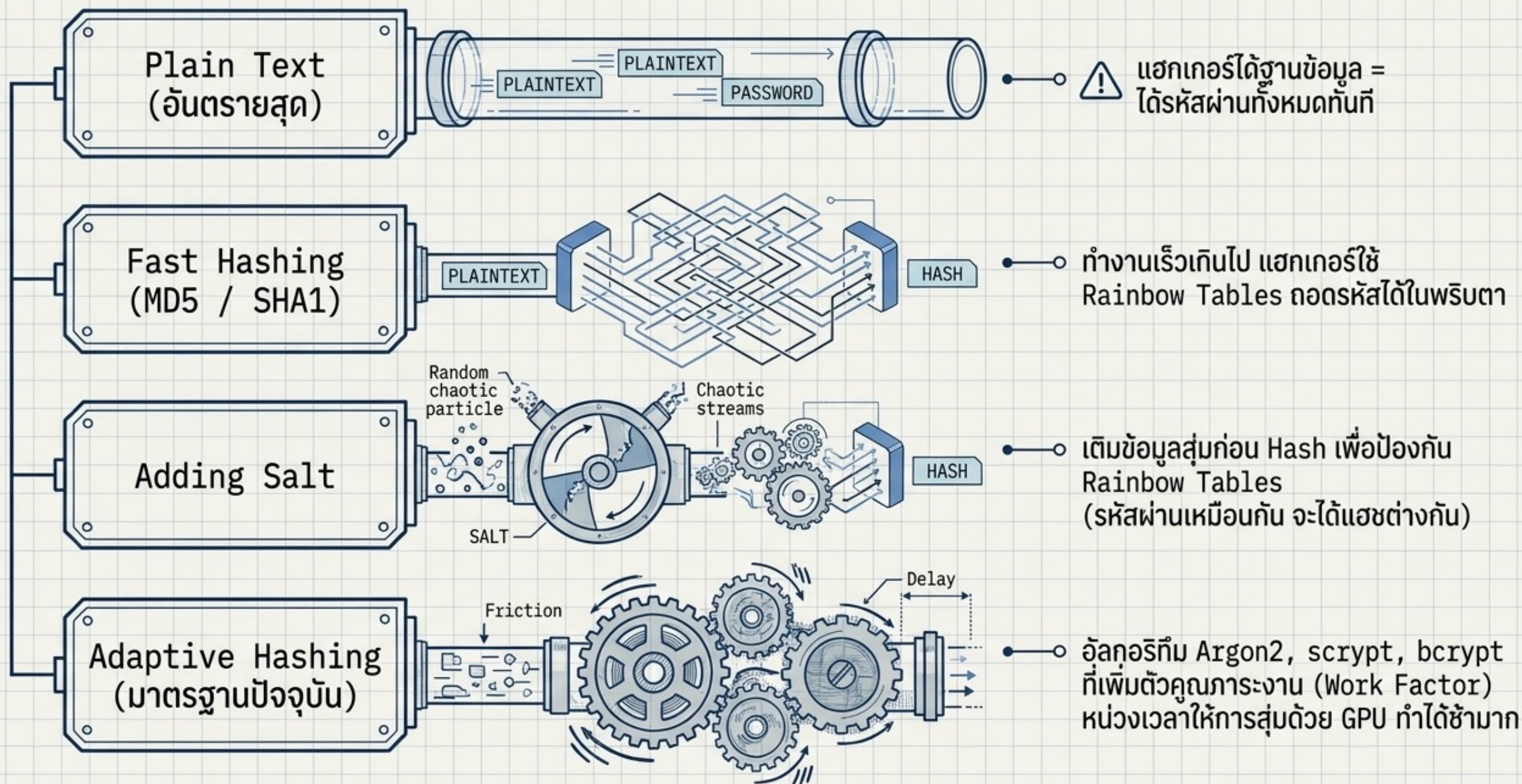


มีการนำระบบการสุ่มมาใช้เพื่อวัตถุประสงค์ทางการเข้ารหัส โดยที่ระบบสุ่มนั้น **ไม่ได้** ถูกออกแบบมาเพื่อให้เป็นไปตามมาตรฐานการเข้ารหัส หรือไม่? และแม้ว่าจะเลือกใช้ฟังก์ชันที่ถูกต้องแล้ว ผู้พัฒนาจำเป็นต้องตั้งค่า **ค่าตั้งต้น (SEED)** เอง หรือไม่? หากไม่ต้อง แล้วผู้พัฒนาได้มีการ **เขียนทับ (OVERWRITE)** ระบบการตั้งค่า **SEED** ที่แข็งแกร่งซึ่งถูกสร้างมากับฟังก์ชันนั้น ด้วยค่า **SEED** ที่ขาดความสุ่มหรือคาดเดาได้ง่ายเกินไปหรือไม่?

- การสุ่มที่คาดเดาได้ = “ไม่มีความปลอดภัย”
- การเลือกฟังก์ชันสุ่มผิดประเภท
 - **Pseudo-Random (PRNG):** เช่น `Math.random()` **คาดเดาได้** หากรู้ค่าตั้งต้น
 - **Cryptographically Secure Pseudo-Random (CSPRNG):** เช่น `crypto.getRandomValues()` หรือ `/dev/urandom` **แฮกเกอร์ไม่สามารถคำนวณย้อนกลับได้**
- ปัญหาเรื่องค่าตั้งต้น (Seed)
 - The Default is Strong
 - Developer Error. ความผิดพลาดมักเกิดจากผู้พัฒนาพยายามจะ “คุมการสุ่ม” เอง โดยการตั้งค่า **Seed** ใหม่ เช่น ใช้ “เวลาปัจจุบัน” (**Timestamp**) เป็น **Seed**



วิวัฒนาการของการจัดเก็บรหัสผ่าน (The Password Storage Evolution)



มีการใช้ฟังก์ชันแฮชที่เสื่อมสภาพหรือล้าสมัยไปแล้ว เช่น **MD5** หรือ **SHA1** หรือไม่? หรือมีการนำฟังก์ชันแฮชที่ไม่ใช่สำหรับการเข้ารหัส มาใช้ในงานที่จำเป็นต้องใช้ฟังก์ชันแฮชสำหรับการเข้ารหัส หรือไม่?

- ปัญหาของฟังก์ชันล้าสมัย (**MD5, SHA1**)
 - สร้าง "การชนกันของข้อมูล" (**Collision**) ได้ง่าย
 - **ความเสี่ยง:** แฮกเกอร์สามารถสร้าง "ไฟล์ปลอม" หรือ "มัลแวร์" ที่มีค่าแฮชเท่ากับ "ไฟล์จริง" เพื่อหลอกให้ระบบยอมรับการติดตั้ง หรือใช้ในการถอดรหัสผ่านด้วยความเร็วสูงผ่าน **GPU**
- การใช้แฮชผิดประเภท (**Non-cryptographic Hash**)
 - เช่น **CRC32** หรือ **MurmurHash** ถูกออกแบบมาเพื่อความเร็วในการตรวจสอบความถูกต้องของข้อมูล (**Checksum**) หรือใช้ในโครงสร้างข้อมูลเพื่อประสิทธิภาพ (**Hash tables**)
 - หากคุณใช้ **CRC32** ในการตรวจสอบความถูกต้องของไฟล์อัปเดตระบบ แฮกเกอร์จะแก้ไขไฟล์และแก้ค่า **CRC32** ให้ตรงกันได้ง่าย ๆ ทำให้ระบบของคุณติดมัลแวร์โดยไม่รู้ตัว



มีการใช้วิธีการเติมข้อมูล (**PADDING METHODS**) ที่ล้าสมัยไปแล้ว เช่น **PKCS#1 V1.5** หรือไม่?

- ในการเข้ารหัสข้อมูล (เช่น **RSA**) ข้อมูลที่เราต้องการส่งมักจะมีความยาวไม่พอดีกับขนาดบล็อกของอัลกอริทึม เราจึงต้องมีการ “เติมข้อมูลส่วนเกิน” (**Padding**) เข้าไปเพื่อให้ครบบล็อก
- ปัญหาของ **PKCS#1 v1.5**
 - **Padding Oracle Attack:** แฮกเกอร์สามารถส่งข้อมูลที่สุ่มขึ้นมาไปยังเซิร์ฟเวอร์ และสังเกต “ข้อความแจ้งเตือนข้อผิดพลาด” (**Error Messages**) ว่า **Padding** ถูกต้องหรือไม่ หากแฮกเกอร์ส่งข้อมูลไปมากพอ เขาจะสามารถถอดรหัสข้อมูลต้นฉบับออกมาได้โดยไม่ต้องรู้กุญแจ (**Private Key**) เลย
- ทำไมถึงยังพบบ่อย?
 - ระบบเก่า ๆ (**Legacy Systems**) มักจะรองรับเฉพาะมาตรฐานนี้
 - **Default Settings:** Library หรือ **SDK** รุ่นเก่าหลายตัวยังตั้งค่ามาตรฐานนี้ไว้เป็นค่าเริ่มต้น (**Default**)



ข้อความแจ้งเตือนข้อผิดพลาดเกี่ยวกับการเข้ารหัส หรือข้อมูลจากช่องทางอื่น สามารถถูกนำไปใช้เพื่อโจมตีระบบได้หรือไม่? ตัวอย่างเช่น การโจมตีในรูปแบบ **PADDING ORACLE ATTACKS**

- แอ็กเตอร์ไม่จำเป็นต้องเจาะเข้าไปในระบบตรง ๆ แต่ใช้วิธี "สังเกตอาการ" ของเซิร์ฟเวอร์เมื่อได้รับข้อมูลที่ผิดปกติ
- **Cryptographic Error Messages** (ข้อความแจ้งเตือนที่บอกมากเกินไป)
- **Side Channel Information** (ข้อมูลจากช่องทางข้างเคียง)
 - **Time (เวลา):** ระบบใช้เวลาตรวจสอบรหัสผ่าน 2 วินาทีถ้าตัวอักษรแรกถูก แต่ใช้เวลาแค่ 0.1 วินาทีถ้าตัวแรกผิด แอ็กเตอร์จะใช้เวลาต่างของเวลานี้เพื่อเดารหัสทีละตัว (**Timing Attack**)
 - **Power/Sound:** การใช้พลังงานของ **CPU** หรือเสียงที่เกิดขึ้นขณะประมวลผลการเข้ารหัส
- **Padding Oracle Attacks** (การโจมตีผ่านคำใบ้การเติมข้อมูล)
 - แอ็กเตอร์ส่งข้อมูลที่เข้ารหัสแบบมั่ว ๆ ไปให้เซิร์ฟเวอร์
 - เซิร์ฟเวอร์พยายามถอดรหัสแล้วพบว่า "**Padding** (ตัวเติมข้อมูล)" ตอนท้ายมันผิดรูปแบบ จึงส่ง **Error** กลับมาว่า "**Padding Error**"
 - แอ็กเตอร์ลองเปลี่ยนข้อมูลไปเรื่อย ๆ จนกว่าเซิร์ฟเวอร์จะเลิกตำเรื่อง **Padding** แต่ไปตำเรื่องอื่นแทน (เช่น "**Data Invalid**")
 - การที่เซิร์ฟเวอร์เปลี่ยนคำตำ ทำให้แอ็กเตอร์รู้ว่าคำที่เขาสุ่มมานั้นมี **Padding** ที่ถูกต้องแล้ว เขาจึงใช้ความรู้นี้คำนวณย้อนกลับหาข้อมูลต้นฉบับ (**Plaintext**) ได้ทีละบล็อจนครบทั้งข้อความ



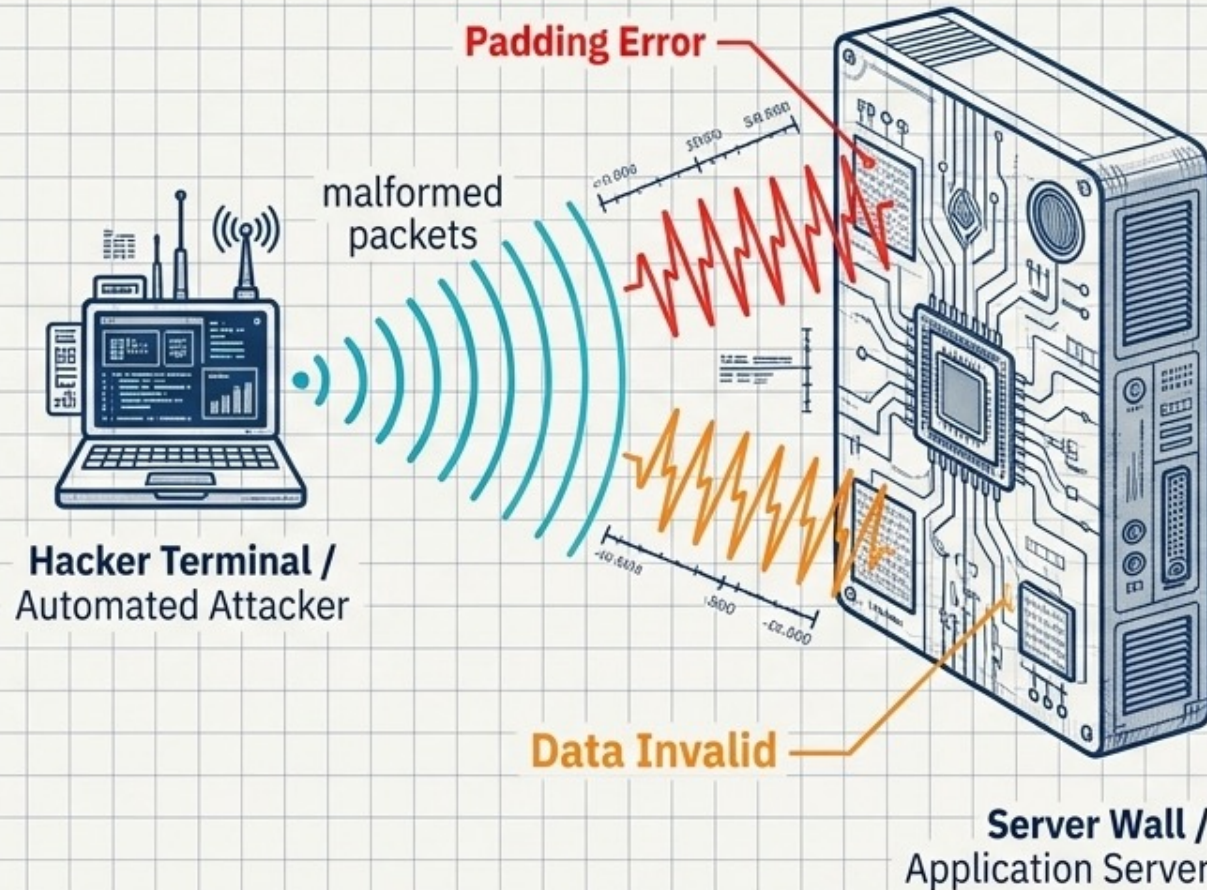
เมื่อคำใบ้กลายเป็นอาวุธ: The Padding Oracle Attack

The Mechanism

ในการเข้ารหัสรุ่นเก่า (PKCS#1 v1.5) ต้องมีการเติมข้อมูล (Padding) ให้เต็มบล็อก

The Attack

แฮกเกอร์ไม่เจาะรหัสผ่าน แต่ส่งข้อมูลสุ่มไปให้เซิร์ฟเวอร์ และสังเกต Error Messages



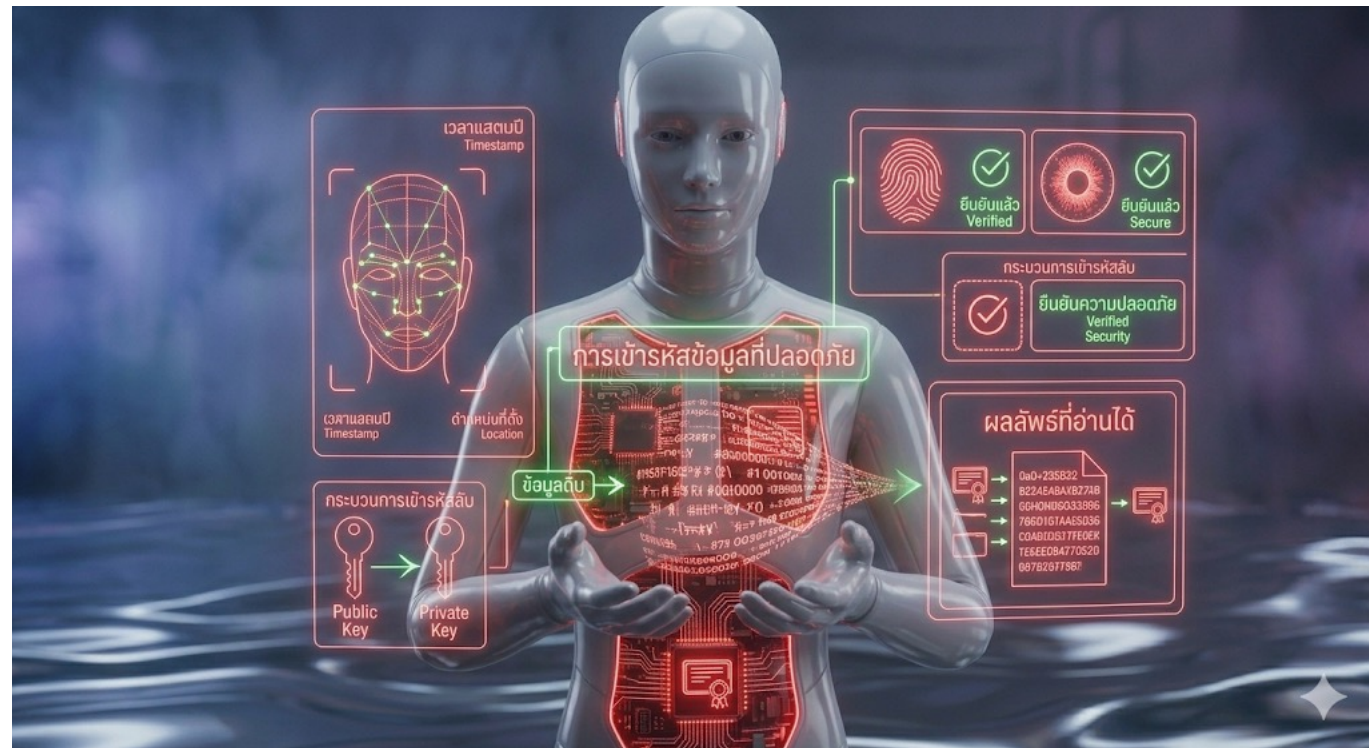
The Sonar Effect

การเปลี่ยนคำต่ำของเซิร์ฟเวอร์ (จาก Padding Error เป็นเรื่องอื่น) ทำให้แฮกเกอร์วิเคราะห์ย้อนกลับเพื่อถอดรหัสข้อความต้นฉบับได้ทีละบล็อก

The Fix

ใช้การเติมข้อมูลรุ่นใหม่ (OAEP) และจัดการ Error ไม่ให้บอกรายละเอียดเชิงลึก

การป้องกัน

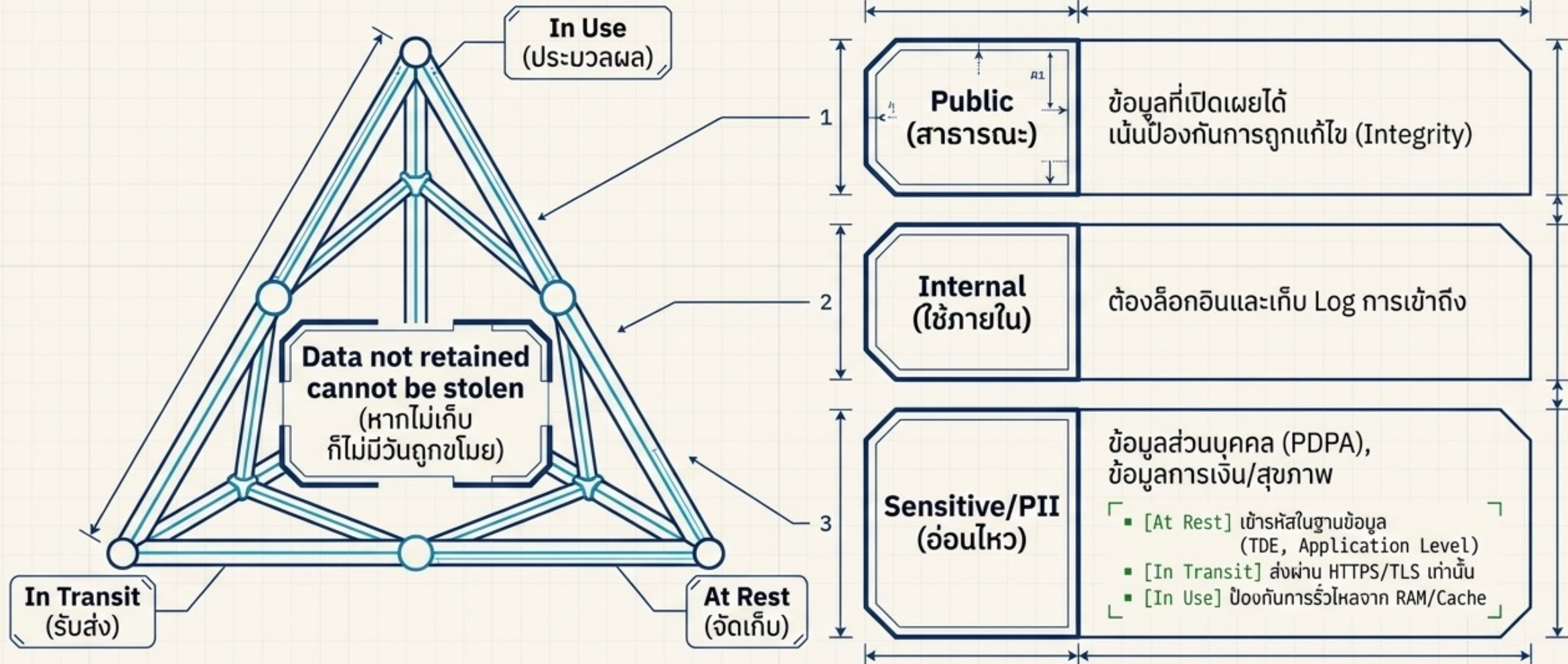


ทำการ **จำแนกประเภทข้อมูล** ทั้งข้อมูลที่ถูกประมวลผล , จัดเก็บ หรือรับส่ง โดยแอปพลิเคชัน พร้อมทั้งระบุว่าข้อมูลใดบ้างที่เป็น **ข้อมูลอ่อนไหว** ตามกฎหมายคุ้มครองความเป็นส่วนตัว, ข้อกำหนดทางระเบียบข้อบังคับ หรือความจำเป็นทางธุรกิจ

- ก่อนที่เราจะป้องกันข้อมูลได้ เราต้องรู้ก่อนว่าเรา "มีอะไรอยู่ในมือบ้าง" และ "อะไรที่สำคัญที่สุด"
- การแบ่งสถานะของข้อมูล (**Data States**)
 - **Data in Use (Processed)**: ข้อมูลที่กำลังถูกใช้งานหรือคำนวณอยู่ในหน่วยความจำ (**RAM**) เช่น ตอนที่ระบบกำลังประมวลผลผล การสอบของนักเรียน
 - **Data at Rest (Stored)**: ข้อมูลที่ถูกเก็บนิ่ง ๆ ใน **Database, Hard drive** หรือ **Cloud** เช่น รายชื่อนักเรียนและเลขบัตรประชาชนที่เก็บไว้ใน **Server**
 - **Data in Transit (Transmitted)**: ข้อมูลที่กำลังวิ่งผ่านเครือข่าย เช่น ตอนที่กด "ส่งข้อมูล" จากหน้าเว็บ
- การจำแนกประเภทข้อมูล (**Data Classification**)
 - **Public** (สาธารณะ): ข้อมูลที่เปิดเผยได้ เช่น ระเบียบการรับสมัคร, ตารางเรียน
 - **Internal** (ใช้ภายใน): ข้อมูลพนักงาน, คู่มือปฏิบัติงาน
 - **Sensitive/Confidential** (อ่อนไหว/ลับ): ข้อมูลที่ถ้าหลุดไปจะเกิดความเสียหาย เช่น เลขประจำตัวประชาชน 13 หลัก, รหัสผ่าน, ประวัติสุขภาพ หรือ ข้อมูลการเงิน
- การระบุตามกฎหมายและระเบียบ (**Compliance**)



เมทริกซ์การจำแนกและสถานะของข้อมูล (The Data Classification Matrix)



อย่าจัดเก็บข้อมูลอ่อนไหวโดยไม่จำเป็น ให้กำจัดทิ้งทันทีที่ใช้งานเสร็จ
หรือใช้ระบบ **TOKENIZATION** ที่ได้มาตรฐาน หรือแม้แต่การตัดข้อมูล
บางส่วนทิ้ง เพราะข้อมูลที่ไม่ได้ถูกเก็บไว้ ย่อมไม่มีวันถูกขโมยได้

- ไม่เก็บข้อมูลโดยไม่จำเป็น (**Avoid Unnecessary Storage**)
- **Tokenization** (การใช้โทเคนแทนข้อมูลจริง)
- **Truncation** (การตัดข้อมูลบางส่วนทิ้ง)
- แทนที่จะเก็บเลขบัตรประชาชนเต็ม ๆ 13 หลัก เราอาจเก็บแค่ 4 หลักสุดท้าย (เช่น **XXXX-XXXXX-12-3**)
- "Data that is not retained cannot be stolen"



ตรวจสอบให้แน่ใจว่าได้ เข้ารหัสข้อมูลอ่อนไหว ทั้งหมดในขณะที่ถูกจัดเก็บ (**AT REST**)

- สิ่งที่ต้องเข้ารหัส “At Rest”
 - ข้อมูลส่วนบุคคล (PII): เลขบัตรประชาชน 13 หลัก, ชื่อ-นามสกุล, ที่อยู่, เบอร์โทรศัพท์ (ตามกฎหมาย PDPA)
 - ข้อมูลประจำตัว: อีเมล, รหัสผ่าน (ซึ่งควรใช้การ **Hashing** แทนการ **Encryption** ทั่วไป)
 - ข้อมูลทางการเงิน: รายละเอียดการชำระเงิน, ประวัติการบัตรเครดิต
 - ไฟล์สำรอง (**Backups**): หลายคนมักลืมเข้ารหัสไฟล์ **Backup** ซึ่งเป็นจุดที่แฮกเกอร์ชอบโจมตีมากที่สุด
- วิธีการดำเนินการ (**Implementation**)
 - **Transparent Data Encryption (TDE)**: เป็นฟีเจอร์ของฐานข้อมูลสมัยใหม่ (เช่น **SQL Server, MySQL**) ที่จะเข้ารหัสข้อมูลให้โดยอัตโนมัติในระดับไฟล์
 - **Disk Encryption**: เข้ารหัสในระดับฮาร์ดแวร์หรือระบบปฏิบัติการ (เช่น **BitLocker**)
 - **Application Level Encryption**: เข้ารหัสเฉพาะฟิลด์ข้อมูลที่สำคัญในโค้ดก่อนจะส่งไปบันทึกในฐานข้อมูล (วิธีนี้ปลอดภัยที่สุด เพราะต่อให้แฮกเกอร์เห็นข้อมูลในฐานข้อมูลก็มองไม่เห็นข้อมูลจริง)



ตรวจสอบให้แน่ใจว่าได้เลือกใช้ อัลกอริทึม, โพรโทคอล และกุญแจ
เข้ารหัส ที่เป็นมาตรฐานสากล มีความแข็งแกร่ง และทันสมัยอยู่เสมอ
รวมถึงต้องมีการ จัดการกุญแจเข้ารหัส ที่เหมาะสมและถูกต้อง

- ความเป็นมาตรฐานและทันสมัย (**Up-to-date & Strong Standard**)
 - **Algorithms:** ห้ามใช้อัลกอริทึมที่ “ติดเองเขียนเอง” (**Home-grown crypto**) เพราะมักจะมีจุดโหว่ที่คาดไม่ถึง ให้เลือกใช้มาตรฐานที่ทั่วโลกยอมรับ เช่น **AES-256** สำหรับการเข้ารหัส หรือ **SHA-256/SHA-3** สำหรับการแฮช
 - **Protocols:** เลือกใช้โพรโทคอลการรับส่งข้อมูลรุ่นล่าสุด เช่น เปลี่ยนจาก **TLS 1.0/1.1** เป็น **TLS 1.2** หรือ 1.3 เพื่อป้องกันช่องโหว่ในระดับเครือข่าย
 - **Keys:** ความยาวของกุญแจต้องเพียงพอต่อการคำนวณของคอมพิวเตอร์ในปัจจุบัน (เช่น กุญแจ **RSA** ควรมีความยาวอย่างน้อย 2048 หรือ 3072 **bits** ขึ้นไป)
- การจัดการกุญแจ (**Proper Key Management**)
 - **Storage:** ห้ามเก็บกุญแจไว้ในโค้ด (**Hardcoded**) แต่ควรเก็บไว้ในอุปกรณ์หรือบริการเฉพาะทาง เช่น **Hardware Security Module (HSM)** หรือ **Key Vault** บนคลาวด์
 - **Access Control:** จำกัดสิทธิ์คนที่เข้าถึงกุญแจให้น้อยที่สุด (**Principle of Least Privilege**)
 - **Rotation:** มีวงจรการเปลี่ยนกุญแจเป็นประจำ (**Key Rotation**) เพื่อลดความเสี่ยงหากกุญแจเดิมถูกกลอบเจาะข้อมูลไปในอดีต



ทำการเข้ารหัสข้อมูลทั้งหมดในขณะรับส่ง ด้วยโปรโตคอลที่ปลอดภัย เช่น **TLS** ที่ใช้ชุดรหัสแบบ **FORWARD SECRECY**, มีการกำหนดลำดับความสำคัญของชุดรหัสโดยเซิร์ฟเวอร์ และตั้งค่าพารามิเตอร์ต่าง ๆ ให้มั่นคงปลอดภัย พร้อมทั้งบังคับใช้การเข้ารหัสด้วยคำสั่งอย่าง **HSTS**

- **Forward Secrecy (FS) คืออะไร?**

- สร้าง “กุญแจชั่วคราว” ขึ้นมาใช้เฉพาะในแต่ละครั้งที่คุยกัน (**Session**)

- **Cipher Prioritization** (การจัดลำดับชุดรหัส)

- เซิร์ฟเวอร์ควรเป็นคนกำหนดว่า “จะคุยกันด้วยภาษาไหนที่ปลอดภัยที่สุด”
- เมื่อเบราว์เซอร์ติดต่อเข้ามา เซิร์ฟเวอร์จะบังคับให้เลือกใช้ชุดรหัสที่แข็งแกร่งที่สุดที่ทั้งสองฝ่ายรองรับก่อนเสมอ แทนที่จะปล่อยให้เบราว์เซอร์เลือกชุดรหัสที่อ่อนแอตามใจชอบ

- **HSTS (HTTP Strict Transport Security)**

- ช่วยป้องกันการโจมตีแบบ **SSL Stripping**



ทำการ ปิดการบันทึกข้อมูลลงหน่วยความจำชั่วคราว สำหรับการตอบรับใด ๆ ที่มีข้อมูลอ่อนไหวรวมอยู่ด้วย

- ทำไมถึงต้องปิด **Cache** สำหรับข้อมูลอ่อนไหว?
 - **Shared Computers:** หากนักเรียนไปใช้คอมพิวเตอร์ที่โรงเรียนหรือร้านเน็ตเพื่อตรวจสอบผลการสอบ แล้วระบบมีการเก็บ **Cache** ไว้ คนที่มาใช้เครื่องต่ออาจสามารถกดปุ่ม "**Back**" (ย้อนกลับ) เพื่อดูข้อมูลส่วนตัวของคนก่อนหน้านี้ได้ แม้ว่าจะกด **Logout** ออกไปแล้วก็ตาม
 - **Local Storage:** ข้อมูลที่ถูก **Cache** มักจะถูกเขียนลงในฮาร์ดดิสก์ของเครื่องนั้น ๆ ในรูปแบบไฟล์ชั่วคราว ซึ่งอาจถูกกู้คืนหรือขโมยได้ง่าย
- วิธีการดำเนินการ (**Implementation**)
 - ต้องตั้งค่าที่ **HTTP Header** ของเซิร์ฟเวอร์ เพื่อสั่งให้เบราว์เซอร์ห้ามจำข้อมูลในหน้านั้นเด็ดขาด
 - **Cache-Control: no-store** (คำสั่งที่สำคัญที่สุด: สั่งว่าห้ามบันทึกข้อมูลลงดิสก์หรือหน่วยความจำใด ๆ)
 - **Cache-Control: no-cache, must-revalidate** (สั่งให้ต้องถามเซิร์ฟเวอร์ใหม่ทุกครั้งก่อนแสดงผล)
 - **Pragma: no-cache** (สำหรับการรองรับเบราว์เซอร์รุ่นเก่า)



นำ มาตรการควบคุมด้านความปลอดภัย ที่จำเป็นมาปรับใช้ ให้ สอดคล้องตามระดับการ จำแนกประเภทข้อมูล ที่กำหนดไว้

■ มาตรการควบคุมคืออะไร? (**Examples of Security Controls**)

- มาตรการทางเทคนิค: เช่น การเข้ารหัส (**Encryption**), การทำ **Masking** (ปิดบางส่วนของข้อมูล), หรือการใช้ระบบ **MFA** เพื่อเข้าถึงข้อมูล
- มาตรการทางกายภาพ: เช่น การเก็บเซิร์ฟเวอร์ไว้ในห้องที่มีการล็อกแน่นหนา หรือการจำกัดคนเข้าห้องดาต้าเซ็นเตอร์
- มาตรการทางบริหารจัดการ: เช่น การกำหนดสิทธิ์เข้าถึงตามหน้าที่งาน (**Role-Based Access Control - RBAC**) หรือการทำข้อตกลงรักษาความลับ (**NDA**) กับพนักงาน

■ การเลือกใช้ตามระดับข้อมูล (**Matching Controls to Class**)

- ข้อมูลสาธารณะ (**Public**): ไม่ต้องเข้ารหัส แต่ต้องป้องกันไม่ให้ใครมาแก้ไขข้อมูลหน้าเว็บได้ (**Integrity**)
- ข้อมูลใช้ภายใน (**Internal**): ต้องมีการล็อกอินเข้าถึง และบันทึก **Log** ว่าใครเข้ามาดูบ้าง
- ข้อมูลอ่อนไหวสูง (**Highly Sensitive** - เช่น เลขบัตรประชาชน/ข้อมูลทางการแพทย์):
 - **At Rest**: ต้องเข้ารหัสในฐานข้อมูล
 - **In Transit**: ต้องส่งผ่าน **HTTPS/TLS** เท่านั้น
 - **Access**: ต้องมีการยืนยันตัวตน 2 ชั้น (**MFA**) และให้เห็นเฉพาะคนที่จำเป็นจริงๆ เท่านั้น



ห้ามใช้โปรโตคอลแบบเก่า เช่น **FTP** และ **SMTP** ในการส่งข้อมูลที่มีความสำคัญ

- **ไม่มีการเข้ารหัส (Lack of Encryption):** โปรโตคอลดั้งเดิมอย่าง **FTP** และ **SMTP** มักจะส่งข้อมูลในรูปแบบ **Plain Text** หรือข้อความธรรมดา หมายความว่าหากมีผู้ไม่หวังดีทำการดักจับข้อมูลระหว่างทาง (**Eavesdropping**) จะสามารถอ่านรหัสผ่านหรือเนื้อหาข้อมูลได้ทันที
- **ความเสี่ยงต่อการถูกโจมตี:** เนื่องจากเป็นเทคโนโลยีเก่า จึงมีช่องโหว่ที่นักจารกรรมข้อมูลรู้จักดี ทำให้เสี่ยงต่อการถูกเจาะระบบได้ง่ายกว่าโปรโตคอลสมัยใหม่
- **แทนที่ FTP:** ให้ใช้ **SFTP** (SSH File Transfer Protocol) หรือ **FTPS** (FTP over SSL/TLS) ซึ่งมีการเข้ารหัสข้อมูลตลอดการเดินทาง
- **แทนที่ SMTP แบบเดิม:** ให้ใช้การส่งผ่านพอร์ตที่รองรับ **STARTTLS** หรือระบบอีเมลที่เข้ารหัสแบบ **End-to-end** เพื่อป้องกันการรั่วไหลของข้อมูล



จัดเก็บรหัสผ่านโดยใช้ฟังก์ชันการแฮชแบบปรับเปลี่ยนได้ที่มีความเข้มแข็ง
และมีการใช้ **SALT** ร่วมกับตัวคูณภาระงาน (**WORK FACTOR** หรือ **DELAY FACTOR**) เช่น **ARGON2**, **SCRYPT**, **BCRYPT** หรือ **PBKDF2**

- **Salted Hashing** (การใช้ Salt): เป็นการเพิ่มข้อความสุ่มสั้น ๆ เข้าไปในรหัสผ่านก่อนจะนำไปเข้ากระบวนการแฮช เพื่อป้องกันการโจมตีแบบ **Rainbow Table** (ตารางรหัสผ่านที่คำนวณไว้ล่วงหน้า) ทำให้ถึงแม้ผู้ใช้สองคนจะตั้งรหัสผ่านเหมือนกัน แต่ค่าแฮชที่บันทึกในฐานข้อมูลจะไม่เหมือนกัน
- **Strong Adaptive Hashing** (การแฮชที่ปรับความแรงได้): หมายถึงอัลกอริทึมที่ออกแบบมาให้ “ช้า” โดยตั้งใจ เพื่อหน่วงเวลาให้การสุ่มเดารหัสผ่านจำนวนมหาศาล (**Brute-force**) ทำได้ยากขึ้นมาก
- **Work Factor / Delay Factor** (ตัวคูณภาระงาน): คือการกำหนดจำนวนรอบในการคำนวณ ยิ่งค่านี้สูง เครื่องคอมพิวเตอร์จะต้องใช้ทรัพยากรและเวลาในการประมวลผลต่อหนึ่งรหัสผ่านมากขึ้น ซึ่งเราสามารถปรับค่านี้ให้สูงขึ้นได้เรื่อย ๆ ตามความแรงของฮาร์ดแวร์ในอนาคต
- **Algorithm** ที่น่าใช้
 - **Argon2**: ผู้ชนะการประกวด **Password Hashing Competition (PHC)** ถือเป็นตัวเลือกที่ดีที่สุดในปัจจุบัน เพราะป้องกันการโจมตีจากทั้ง **CPU** และ **GPU** ได้ดีเยี่ยม
 - **scrypt**: ออกแบบมาเพื่อใช้หน่วยความจำ (**Memory**) สูง เพื่อป้องกันการใช้เครื่องชุดบิตคอยน์หรือชิปเฉพาะทาง (**ASIC**) มาช่วยถอดรหัส
 - **bcrypt**: เป็นมาตรฐานที่ใช้กันมานานและมีความเสถียรสูงมาก ปรับแต่งความยากได้ผ่านค่า “**Cost**”
 - **PBKDF2**: เป็นมาตรฐานเก่าแก่ที่ยังคงใช้งานได้ดีและมักถูกกำหนดไว้ในมาตรฐานความปลอดภัยระดับองค์กรหลายแห่ง



การเลือกค่าตัวแปรเริ่มต้น (**IV**) ต้องเลือกให้เหมาะสมกับโหมดการทำงาน สำหรับหลายโหมดนั้นหมายถึงการใช้ **CSPRNG** ส่วนโหมดที่ต้องการเพียง **NONCE** ค่า **IV** ก็ไม่จำเป็นต้องใช้ **CSPRNG** อย่างไรก็ตาม ไม่ว่าจะกรณีใดก็ตาม ห้ามใช้ค่า **IV** ซ้ำกันสองครั้งกับกุญแจรหัสเดิม

- **Initialization Vector (IV)** คืออะไร?

- ตัวเลขสุ่มที่ถูกนำมาผสมกับข้อมูลเริ่มต้นก่อนที่จะเริ่มกระบวนการเข้ารหัส เพื่อให้แน่ใจว่าแม้จะมีการเข้ารหัสข้อความเดิมซ้ำด้วยกุญแจรหัสเดิม ผลลัพธ์ที่ได้ (**Ciphertext**) จะต้องออกมาไม่เหมือนเดิมทุกครั้ง เพื่อป้องกันไม่ให้ผู้โจมตีสังเกตเห็นรูปแบบของข้อมูลได้

- **CSPRNG vs. Nonce**

- **CSPRNG**: เป็นอัลกอริทึมการสุ่มที่ซับซ้อนมากจนไม่สามารถคาดเดาตัวเลขถัดไปได้เลย เหมาะสำหรับโหมดการเข้ารหัสที่ต้องการความเป็นสุ่มสูง
- **Nonce (Number used once)**: คือตัวเลขที่เน้นแค่ “ห้ามซ้ำ” เท่านั้น เช่น การใช้เลขลำดับ (**Counter**) หรือการใช้เลขเวลา ซึ่งไม่จำเป็นต้องคาดเดายาก แต่ต้องมั่นใจว่าจะไม่เกิดขึ้นซ้ำในการเข้ารหัสครั้งต่อไป



ใช้การเข้ารหัสแบบยืนยันตัวตนเสมอ แทนที่จะใช้ เพียงแต่การเข้ารหัสเพียงอย่างเดียว

- **Encryption** (การเข้ารหัสทั่วไป): ทำหน้าที่ “รักษาความลับ” (**Confidentiality**) คือทำให้คนอื่นอ่านไม่ออก แต่มีจุดอ่อน คือ หากผู้โจมตีแอบแก้ไขข้อมูลที่ถูกเข้ารหัสไว้ (แม้จะอ่านไม่ออกก็ตาม) เมื่อนำข้อมูลนั้นมาถอดรหัส เราอาจจะได้ข้อมูลที่ผิดเพี้ยนไปจากเดิม ซึ่งอาจส่งผลกระทบต่อระบบได้
- **Authenticated Encryption** (การเข้ารหัสแบบยืนยันตัวตน): ทำหน้าที่ทั้ง “รักษาความลับ” และ “ตรวจสอบความถูกต้อง” (**Integrity**) ไปพร้อมกัน โดยระบบจะมีการสร้างค่าตรวจสอบ (**Tag** หรือ **MAC**) แนบไปกับข้อมูลด้วย
- อัลกอริทึมที่แนะนำ
 - **AES-GCM (Galois/Counter Mode)**: เป็นมาตรฐานที่นิยมที่สุดในปัจจุบัน รวดเร็วและปลอดภัยสูง
 - **ChaCha20-Poly1305**: เป็นทางเลือกที่ดีมากสำหรับอุปกรณ์ที่ไม่มีชิปช่วยคำนวณ **AES** (เช่น อุปกรณ์ **IoT** หรือสมาร์ทโฟนบางรุ่น)



กุญแจรหัสควรถูกสร้างขึ้นด้วยวิธีการสุ่มทางวิทยาการรหัสลับ และจัดเก็บไว้ในหน่วยความจำในรูปแบบของ **BYTE ARRAYS** หากมีการใช้รหัสผ่าน รหัสผ่านนั้นจะต้องถูกเปลี่ยนให้เป็นกุญแจรหัสผ่านฟังก์ชันการอนุพันธ์ กุญแจจากรหัสผ่าน ที่เหมาะสม

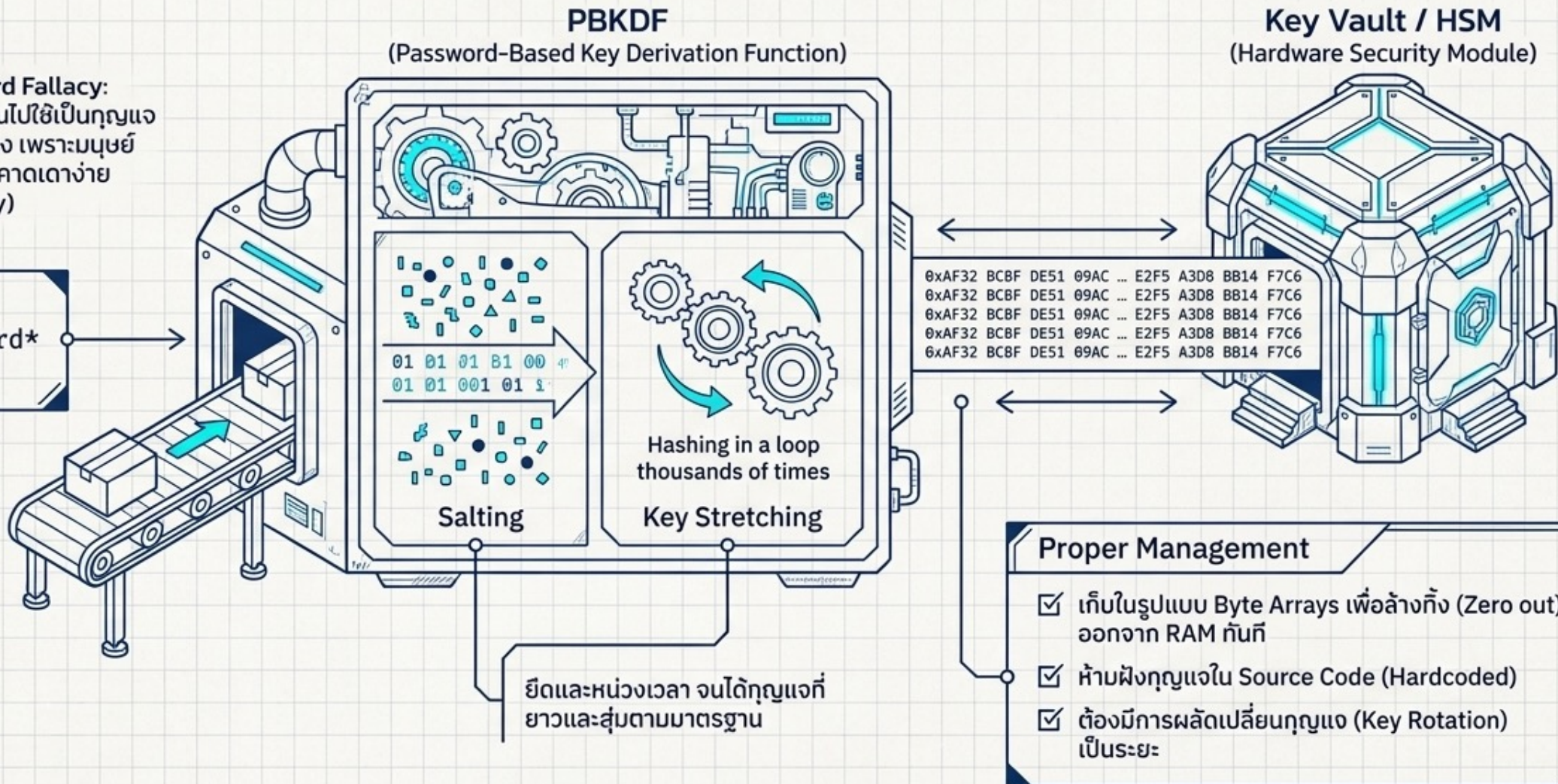
- **Cryptographically Random Generation:** กุญแจรหัสที่ดีต้อง “สุ่ม” อย่างแท้จริงจนไม่มีใครคาดเดาได้ โดยต้องใช้ฟังก์ชันสุ่มระดับสูง (เช่น `/dev/urandom` ใน **Linux** หรือ **Secrets** ใน **Python**) ไม่ใช่การสุ่มแบบธรรมดาที่ใช้ในเกมหรือการเขียนโปรแกรมทั่วไป
- **Stored as Byte Arrays:** การเก็บกุญแจในรูปแบบ **Byte Arrays** (ชุดของตัวเลขฐานสอง) ดีกว่าการเก็บเป็น **String** (ข้อความ) เพราะ **String** ในหลายภาษาโปรแกรมมักจะถูกเก็บค้างไว้ในหน่วยความจำนานเกินไปและจัดการลบออกได้ยาก การใช้ **Byte Array** ช่วยให้เราสามารถ “ล้างทิ้ง” (**Zero out**) ข้อมูลกุญแจออกจาก **RAM** ได้ทันทีเมื่อเลิกใช้งาน ลดความเสี่ยงหากเครื่องคอมพิวเตอร์ถูก **Dump** หน่วยความจำ
- **Key Derivation Function (KDF):** นี่คือจุดที่สำคัญมากครับ ปกติรหัสผ่านที่คนตั้ง (เช่น **MyP@ssw0rd**) จะมีความยาวและรูปแบบที่ไม่เหมาะสมจะนำไปใช้作为กุญแจรหัสโดยตรง เราจึงต้องส่งรหัสผ่านนั้นเข้าเครื่องบดที่เรียกว่า **KDF** (เช่น **PBKDF2**, **scrypt** หรือ **Argon2**) เพื่อยืด (**Stretch**) และผสมค่าสุ่ม (**Salt**) จนออกมาเป็นกุญแจรหัสที่มีความเข้มข้นสูง



กายวิภาคของการสร้างและจัดการกุญแจ (Cryptographic Key Management)

! The Password Fallacy:
ห้ามนำรหัสผ่านไปใช้เป็นกุญแจ
เข้ารหัสโดยตรง เพราะมนุษย์
ตั้งรหัสสั้นและคาดเดาง่าย
(Low Entropy)

The Password



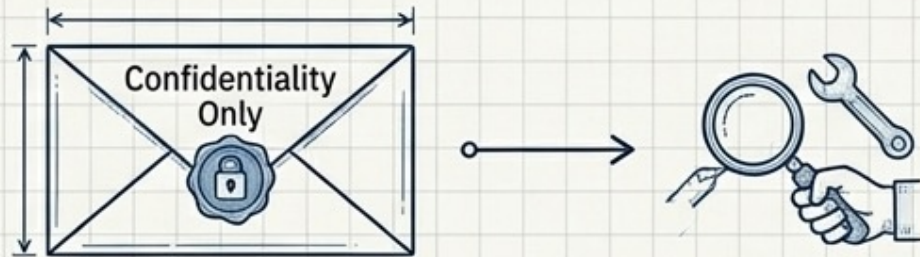
จงมั่นใจว่าได้ใช้การสุ่มทางวิทยาการรหัสลับ ในส่วนที่เหมาะสม และต้องไม่มี การตั้งค่าเริ่มต้น ในรูปแบบที่คาดเดาได้หรือมีความโกลาหลต่ำ ซึ่งโดยส่วนใหญ่แล้ว **API** สมัยใหม่ไม่จำเป็นต้องให้ผู้พัฒนาทำการ **SEED** ค่าให้กับ **CSPRNG** ด้วยตัวเองเพื่อให้เกิดความปลอดภัย

- **Cryptographic Randomness vs. Standard Random:**
- การสุ่มแบบทั่วไป (เช่น `rand()` หรือ `Math.random()`) ออกแบบมาเพื่อความเร็วและเน้นการกระจายตัวของตัวเลข แต่ คาดเดาได้ หากรู้ค่าเริ่มต้น (**Seed**)
- การสุ่มทางรหัสลับ (**CSPRNG**) ออกแบบมาให้ คาดเดาไม่ได้เลย แม้จะรู้ผลลัพธ์ย้อนหลังไปมากเท่าไรก็ตาม เหมาะสำหรับสร้างกุญแจ รหัส, **IV**, **Salt** หรือ **Session ID**
- **The Problem with Seeding** (ปัญหาของการตั้งค่าเริ่มต้น):
 - ในสมัยก่อน ผู้พัฒนาต้องหาค่าสุ่มมาเป็น “เชื้อ” (**Seed**) ให้กับฟังก์ชันสุ่ม เช่น การใช้เวลาปัจจุบัน (**Timestamp**)
 - หากใช้ค่าที่คาดเดาง่ายอย่างเวลา หรือค่าที่มีความโกลาหล (**Entropy**) ต่ำ ผู้โจมตีสามารถสุ่มหาค่า **Seed** นั้น (**Brute-force**) และจะรู้ผลลัพธ์ การสุ่มทั้งหมดที่ตามมาทันที
- **Modern APIs:**
 - ปัจจุบันภาษาโปรแกรมและระบบปฏิบัติการสมัยใหม่ (เช่น **Python secrets**, **Java SecureRandom**, หรือ `/dev/urandom` ใน **Linux**) จะดึงค่าความโกลาหลจากฮาร์ดแวร์และสภาพแวดล้อมมาจัดการเรื่อง **Seed** ให้โดยอัตโนมัติ
 - ดังนั้น หน้าทีของคุณคือ “เรียกใช้ฟังก์ชันให้ถูกต้อง” และ “ไม่ต้องพยายามไปตั้งค่า **Seed** เอง” เว้นแต่จะมีเหตุผลทางเทคนิคที่เฉพาะเจาะจงจริงๆ



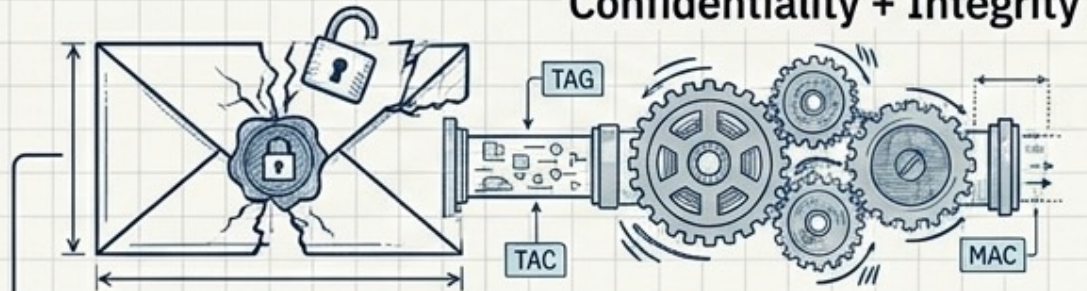
มาตรฐานความปลอดภัยยุคใหม่: Authenticated Encryption & Randomness

Authenticated Encryption (เช่น โหมด AES-GCM)



การเข้ารหัสปกติช่วยแค่อ่านไม่ออก
แต่อาจถูกดัดแปลง

Confidentiality + Integrity



การเข้ารหัสแบบมีตัวตรวจสอบ (Tag/MAC)
หากถูกดัดแปลงแม้แค่บิตเดียว
ระบบจะปฏิเสธข้อมูลนั้นทันที

0
0
0 → 1
1
REJECT

Cryptographically Secure Random (CSPRNG)



PRNG / Timestamp Seed
(คาดเดาได้หากรู้เวลาตกฟาก)

Predictable Sequence
if Seed/Time is Known



CSPRNG
(คาดเดาไม่ได้เลย)

True Entropy &
Unpredictable Stream



API สมัยใหม่ดึงค่าความโกลาหลมาจัดการ Seed ให้เอง
ผู้พัฒนาไม่ควรตั้งค่า Seed ด้วยตัวเอง

หลีกเลี่ยงการใช้ฟังก์ชันทางวิทยาการรหัสลับและรูปแบบการเติมข้อมูล ที่เสื่อมสภาพแล้ว เช่น **MD5, SHA1** และ **PKCS #1 V1.5**

- ฟังก์ชันที่เสื่อมสภาพ (**Deprecated Functions**)
 - **MD5** และ **SHA1**: ทั้งสองเป็นฟังก์ชันแฮช (**Hash Functions**) ที่ปัจจุบันถูกพิสูจน์แล้วว่ามีความเสี่ยงเรื่อง **Collision** (การชนกันของข้อมูล) หมายความว่าผู้โจมตีสามารถสร้างข้อมูลสองชุดที่แตกต่างกันแต่มีค่าแฮชเหมือนกันได้ ทำให้การตรวจสอบความถูกต้องของไฟล์หรือลายเซ็นดิจิทัลเชื่อถือไม่ได้ต่อไป
 - คำแนะนำ: ควรเปลี่ยนไปใช้ **SHA-256** หรือ **SHA-3** แทน ซึ่งมีความแข็งแกร่งกว่ามาก
- รูปแบบการเติมข้อมูลที่เสื่อมสภาพ (**Deprecated Padding**)
 - **Padding** คือการเติมข้อมูลหลอกเข้าไปเพื่อให้ขนาดของข้อมูลเหมาะสมกับอัลกอริทึมการเข้ารหัส
 - **PKCS #1 v1.5**: เป็นรูปแบบการเติมข้อมูลรุ่นเก่าสำหรับ **RSA** ที่มีช่องโหว่ต่อการโจมตีแบบ **Padding Oracle Attack** ซึ่งช่วยให้ผู้โจมตีสามารถถอดรหัสข้อมูลได้จากการสังเกตข้อความตอบกลับของระบบ
 - คำแนะนำ: ควรเปลี่ยนไปใช้ **OAEP** (**Optimal Asymmetric Encryption Padding**) สำหรับการเข้ารหัส และ **PSS** (**Probabilistic Signature Scheme**) สำหรับการทำลายเซ็นดิจิทัลแทน



เมทริกซ์มาตรฐานการเข้ารหัส (Cryptographic 'Do This, Not That')

เลิกใช้ (Deprecated)	มาตรฐานที่ควรใช้ (Adopt)
MD5, SHA1, CRC32	SHA-256, SHA-3
Plaintext, Simple Hash	Argon2, scrypt, bcrypt, PBKDF2 (มี Salt + Work Factor)
DES, 3DES, RC4	AES-GCM, ChaCha20-Poly1305
PKCS#1 v1.5	OAEP (เข้ารหัส), PSS (ลายเซ็นดิจิทัล)
SSL v2/v3, TLS 1.0/1.1, FTP, SMTP	TLS 1.2 / 1.3, SFTP, กำหนด HSTS
PRNG, ตั้ง Seed เองด้วย Timestamp	CSPRNG (/dev/urandom) ปล่อย API จัดการ Seed

ให้ทำการตรวจสอบประสิทธิภาพของการ กำหนดค่าและการตั้งค่าต่าง ๆ อย่างเป็นอิสระ

- ทำ "**Audit**" หรือการตรวจทานซ้ำเพื่อให้มั่นใจว่า ระบบความปลอดภัยที่เราตั้งค่าไว้นั้นทำงานได้จริงตามที่ออกแบบไว้
- **Verify Independently** (ตรวจสอบอย่างเป็นอิสระ): หมายถึงการให้บุคคลที่สาม หรือใช้เครื่องมืออื่นที่ไม่ใช่ตัวระบบเองมาเป็นผู้ตรวจสอบ ตัวอย่างเช่น หากคุณเป็นคนเขียนโค้ดตั้งค่าความปลอดภัยของฐานข้อมูลในระบบ คุณควรให้เพื่อนร่วมงานท่านอื่น หรือทีม **Security** มาช่วยตรวจสอบ (**Peer Review**) หรือใช้เครื่องมืออัตโนมัติมา **Scan** หาช่องโหว่
- **Effectiveness** (ประสิทธิภาพ): ไม่ใช่แค่เช็กว่า "ตั้งค่าหรือยัง" (**Checklist**) แต่ต้องเช็กว่า "ตั้งค่าแล้วกันได้จริงไหม" เช่น ตั้งค่า **Firewall** ไว้แล้ว ก็ต้องลองยิงทดสอบดูว่ามัน **Block** ได้จริงตามเงื่อนไขหรือไม่
- **Configuration and Settings**: ครอบคลุมตั้งแต่การตั้งค่า **Server**, ฐานข้อมูล **SQL**, การตั้งค่า **IV**, ไปจนถึงการจัดการสิทธิ์การเข้าถึงข้อมูล (**Permissions**) ในระบบ



ตัวอย่างสถานการณ์การโจมตี (ATTACK SCENARIOS)

- สถานการณ์ที่ 1: การถอดรหัสอัตโนมัติที่กลายเป็นช่องโหว่
 - แอปพลิเคชันมีการเข้ารหัสหมายเลขบัตรเครดิตในฐานข้อมูลโดยใช้ระบบเข้ารหัสฐานข้อมูลแบบอัตโนมัติ อย่างไรก็ตาม ข้อมูลนี้จะถูกถอดรหัสคืนโดยอัตโนมัติเมื่อมีการเรียกใช้งาน ซึ่งทำให้เมื่อเกิดช่องโหว่ประเภท **SQL Injection** ผู้โจมตีจึงสามารถดึงข้อมูลหมายเลขบัตรเครดิตออกมาเป็นข้อความปกติ (**Clear Text**) ได้ทันที
- สถานการณ์ที่ 2: การดักจับข้อมูลและการจารกรรมเซสชัน (**Session Hijacking**)
 - เว็บไซต์ไม่ได้บังคับใช้ **TLS (HTTPS)** ในทุกหน้า หรือรองรับการเข้ารหัสที่อ่อนแอ ผู้โจมตีทำการเฝ้าสังเกตปริมาณข้อมูลในเครือข่าย (เช่น ในวง **Wi-Fi** ที่ไม่ปลอดภัย) แล้วทำการลดระดับการเชื่อมต่อจาก **HTTPS** ลงมาเป็น **HTTP (Downgrade Attack)** เพื่อดักจับคำขอและเซสชัน **Session Cookie** ของผู้ใช้ จากนั้นผู้โจมตีจะนำ **Cookie** นี้ไปใช้งานซ้ำ (**Replay**) เพื่อปลอมแปลงเป็นผู้ใช้นั้นเข้าถึงหรือแก้ไขข้อมูลส่วนตัว หรือแม้กระทั่งเปลี่ยนแปลงข้อมูลระหว่างการส่ง เช่น เปลี่ยนชื่อผู้รับเงินโอน
- สถานการณ์ที่ 3: การใช้ฟังก์ชันแฮชที่อ่อนแอและไม่มี **Salt**
 - ฐานข้อมูลรหัสผ่านใช้การแฮชแบบธรรมดาหรือไม่มีการใช้ **Salt** ในการเก็บรหัสผ่านของทุกคน เมื่อผู้โจมตีใช้ช่องโหว่จากการอัปโหลดไฟล์ถึงไฟล์ฐานข้อมูลรหัสผ่านไปได้ รหัสผ่านที่แฮชโดยไม่มี **Salt** ทั้งหมดจะถูกเปิดเผยด้วยการใช้ **Rainbow Table** (ตารางคำนวณแฮชล่วงหน้า) นอกจากนี้ แฮชที่สร้างจากฟังก์ชันที่ทำงานเร็วเกินไป อาจถูกถอดรหัสได้ด้วยการใช้ **GPU** แม้ว่าจะมีการใช้ **Salt** ร่วมด้วยก็ตาม



กายวิภาคของการเจาะระบบ (Real-World Attack Chains)

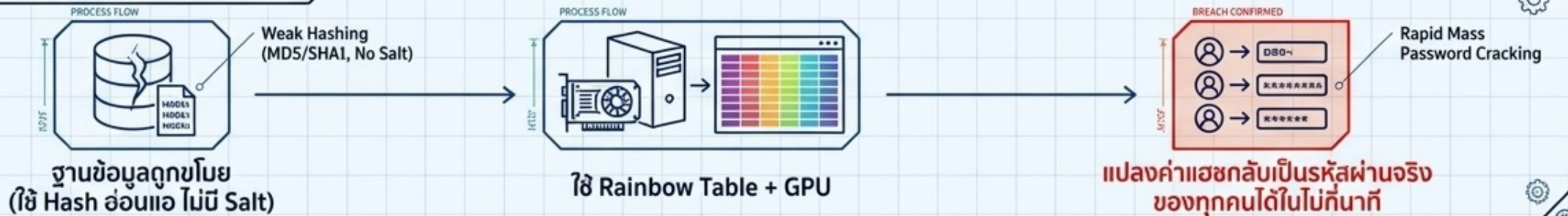
Scenario 1: Automated Decryption via SQLi



Scenario 2: Session Hijacking via Downgrade



Scenario 3: DB Leak to Rainbow Table



บทสรุปสถาปัตยกรรม (The Digital Fortress Architecture)

Pillar 1: MFA & Rate Limiting

ปิดประตูการโจมตีแบบอัตโนมัติ
(Credential Stuffing / Brute Force)
ในทุกจุดเข้าถึง



Pillar 2: Adaptive Password Hashing

ปกป้องฐานข้อมูลรหัสผ่านด้วย
Algorithm ที่หน่วงเวลาได้
(Argon2) และใส่ Salt เสมอ



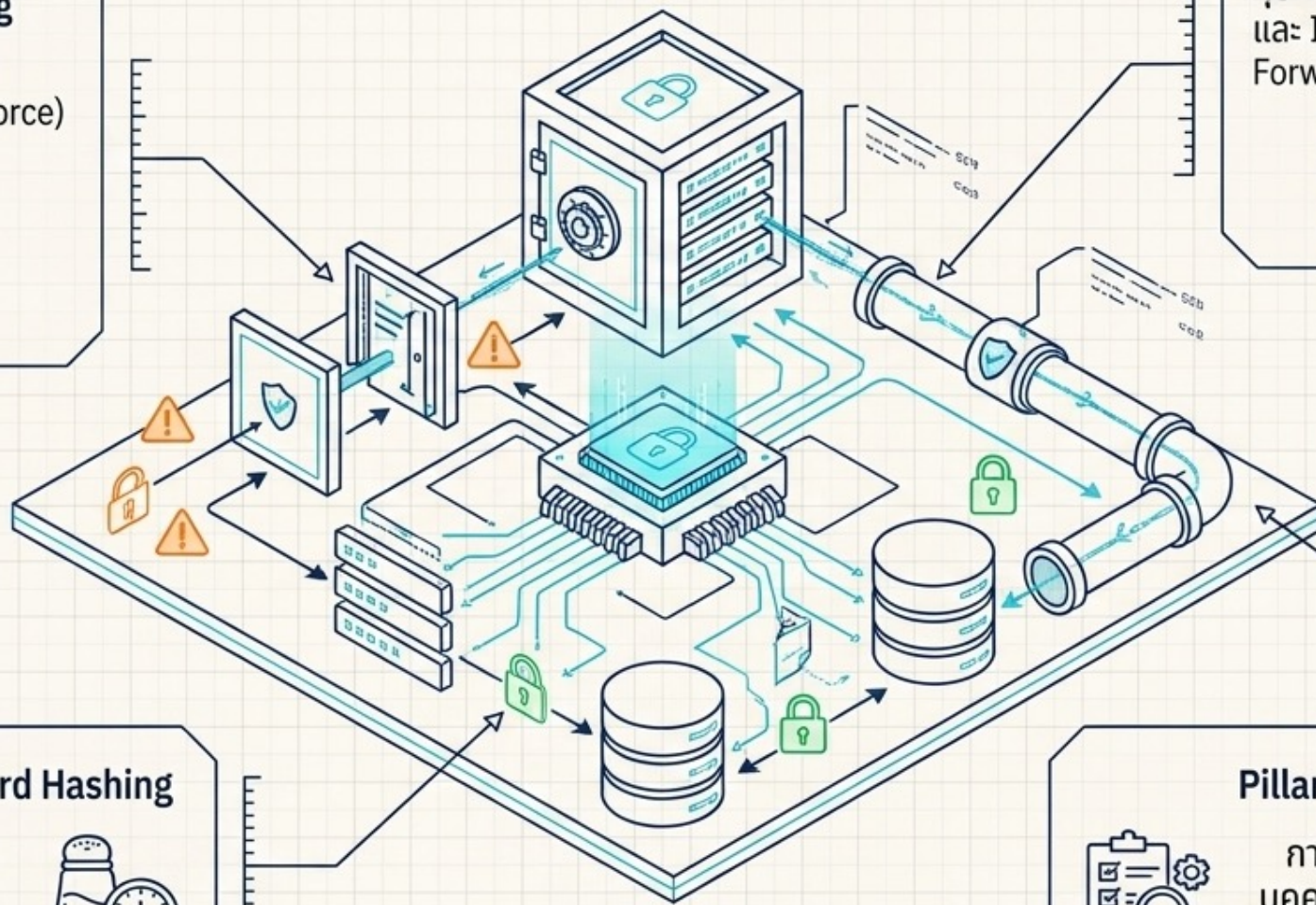
Pillar 3: Authenticated Encryption

คุ้มครองข้อมูลทั้ง At Rest (AES-GCM)
และ In Transit ด้วยชุดรหัสแบบ
Forward Secrecy



Pillar 4: Verify Independently

การตั้งค่าต้องถูก Audit ชำจาก
บุคคลที่สามหรือเครื่องมืออัตโนมัติ
เพื่อให้ระบบทำงานได้จริงตามพิมพ์เขียว





ระบบความปลอดภัยทางเว็บ
Web Security System

Thanks

ขอบคุณ